BCSC0007: INTRODUCTION TO MICROPROCESSORS

Objective: *Objective of this subject is to introduce the basic concepts of microprocessor and assembly language programming. Identify and explain the operation of the components of typical microprocessor: the role of the ALU, registers, stack and the use of interrupts.*

Credits: 03 **L–T–P-J: 3–0–0-0**

Module No.	Content	Teaching Hours
I	**Introduction**: Microprocessors Evolution and Types, Basics of Pentium Microprocessor, Microprocessor Application, **8-Bit Microprocessor:**8085 Microprocessor and its Architecture, Addressing Modes, The 8085 Programming Model, Instruction Classification, Instruction Format, Overview of Instruction Set - Data Transfer Operation, Arithmetic Operation, Logic Operations and Branch Operations; Introduction to Assembly Language Program. **Programming Technique with Additional Instruction**: Looping, Counting, Indexing, Additional Data Transfer and 16-Bit Arithmetic Instruction, Counters and Time Delays, Stack and Subroutine.	20
II	**16 Bit Microprocessor:** Architecture of 8086 – Register Organization, Execution Unit, Bus Interface Unit, Signal Description, Physical Memory Organization, Mode of Operation, I/O Addressing Capabilities. **Peripheral Interfacing:** I/O Programming, Programmed I/O, Interrupt Driven I/O, DMA I/O, Memory-Mapped I/Os. **Peripheral Devices:** 8237 DMA Controller, 8255 Programmable Peripheral Interface, 8253/8254 Programmable Timer/Counter, 8259 Programmable Interrupt Controller.	18

Text Books:

- N Senthil Kumar, MSaravanan, and S Jeevananthan, "Microprocessors and Microcontrollers", Oxford University Press India, 2010.

Reference Books:

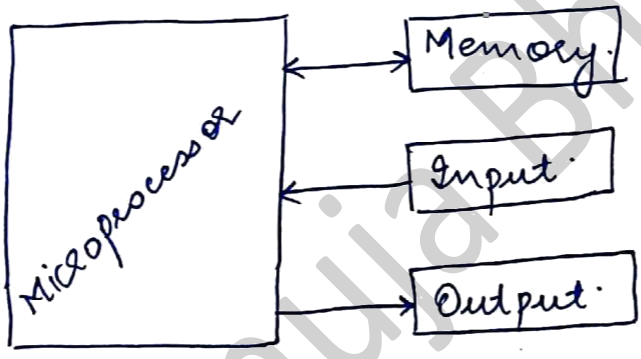
- Ramesh S. Gaonkar , "Microprocessor Architecture Programming and Applications with 8085", 4th Edition, Penram International Publishing, 2000.
- Ray A.K. Bhurchandi.K.M, "Advanced Microprocessor and Peripherals", TMH, 2002.
- D. V. Hall, "Microprocessors and Interfacing: Programming and Hardware", 2nd Edition, TMH, 1992.
- Y.C. Liu and G.A. Gibson, "Microcomputer Systems: The 8086/8088 Family Architecture Programming and Design", 2nd Edition, PHI, 2003.

Outcome: After the completion of the course, the student will be able to:

- CO1: Demonstrate the Microprocessor internal architecture and its operations.
- CO2: Develop programs based on 8085 microprocessor instruction set and addressing mode.
- CO3: Develop program using looping, counting, indexing, counter and time delays.
- CO4: Understand the concept of stack and subroutine for modular approach.
- CO5: Compare accepted standards and guidelines to select microprocessor (8085 & 8086) to meet performance requirements.
- CO6: Analyze the concept of interfacing the processor to external device with I/O programming & Interrupt Driven I/O.
- CO7: Understand the working of interfacing chips (8237, 8253/54, 8255 & 8259).

Microprocessor :-

- A microprocessor is a multipurpose, programmable, clock driven, register based electronic device that reads binary instruction from a storage device called Memory, accepts binary data as input and processes data according to those instructions, and provide result as output.
- A typical programmable machine can be represented with four components : Microprocessor, Memory, Input & Output.



- Example of programmable machine
  - → Turn traffic light ON / OFF.
  - → Compute mathematical function
  - → Washing Machine, Dish washer
  - → Dashboard Controls

Dr Anuja Bhargava

Anuja

- **Memory** - is like the page of Notebook with space for fixed number of binary number on each line. These pages are made of semiconductor Material. Typically, each line is an 8-bit register that can store eight binary bits and several of these registers are stored in a sequence called memory. These registers are always group together in power of two.

  Example: A group of 1024 (2^{10}) 8 bit register on a semiconductor chip is known as 1K byte of memory.

- **History**

Processor	Year	Transistor Used	Clock Speed	Address Bus	Data Bus
8085	1976	6500	5 MHz	16-bit	8-bit
8086	1978	29000	5 MHz	20-bit	16-bit

- Why Name as 8085 ?
  → Generated in 1976 (80)
  → Processor is 8-bit
  → Work at 5 Volt

- Why 8086 ?
  → Next version of 8085.

Dr. Anuja Bhargava

- Memory stores binary information as instructions and data and provides that information to the processor whenever necessary.
- Memory has two sections
  - → ROM (Read Only Memory)

    It is used to store programs that do not need alterations. This program interprets the information entered through keyboard and provide equivallent binary digit to microprocessor.
  - → RAM (Random Access Memory)

    It is used to store user programs and data. The information stored in this memory can be easily read and altered.
- Input / Output :- It communicate with outside world also known as peripherals.
- System - Bus - It is a communication path between microprocessor and peripherals. It is nothing but a group of wires to carry bits. All peripherals share the same bus, the microprocessor communicates with one peripheral at a time.

Anuja

Dr. Anuja Bhargava

• How does microprocessor work?

→ The instructions are stored sequentially in the memory.

→ The microprocessor fetch the first instruction from the Memory, decodes it and executes that instruction.

→ The sequence of fetch, decode and execute is continue until the microprocessor come across an instruction to stop.

→ During the entire process, the microprocessor uses the system bus to fetch the binary instructions and data from memory.

→ It uses register to store data and perform computing function in ALU.

→ Finally, it sends the result in binary, using the same bus lines, to the seven segment LED's.

• ASCII Code.

→ A computer is a binary machine, to communicate with computer in alphabetic letters and decimal numbers translation code are necessary.

→ The commonly code is known as ASCII - American Standard Code for Information Interchange.

→ It is a 7-bit code with 128 (2^7) combinations. and each code is assigned as letter, alphabet, decimal number or symbol.

Dr. Anuja Bhargava

Evolution of Microprocessor :-

- The history of microprocessor starts from 1971. The first µP launched by Intel in 1971 i.e Intel 4004, a 4-bit microprocessor.
- The 4004 consist of ALU and control unit. It provides only 45 instructions. It is a 16 pin dual in-line package integrated circuit.
- After 4004 various other 4 bit microprocessor are launched in the market by other manufactures. eg Intel 4040, Rockwell International PPS4, Toshiba T3472.
- In 1972, the first 8 bit microprocessor, 8008, launched by Intel. After this, 8080, 8 bit µP is introduced which is enhanced version of 8008. These are designed by using PMOS technology. These
- are basically first generation µP.
- After 1973, the second generation processor, known as 8 bit µP such as Motorola 6800, & 6809, Intel 8085, Zilog Z80 and Fairchild F8 are launched by different manufacturers. These are designed by using NMOS technology.

Anuja

Dr. Anuja Bhargava

Microprocessor Name	Manufacturer	Distinction
4004	Intel	First μp (1971)
8008	Intel	First 8-bit μp (1972)
8080 A	Intel	First n-channel, second generation μp (1974)
6800	Motorola	First +5V only μp (1974)
PACE	National Semiconductor	First 16-bit μp (1974)
1802	RCA	First CMOS μp (1974)
8048	Intel	First 8-bit single-chip Microcomputer (1976)
8088	Intel	First 8-bit processor with 16-bit internal architecture (1979)
80386	Intel	First 32-bit μp (1982)
Pentium	Intel	First 64-bit μp (1993)

Dr. Anuja Bhargava

- After 1978, the third generation µp known as 16-bit microprocessor are came into the market.

eg Intel 8086, Motorola M68000, Zilog's Z8000,

- These µp were designed using High Density MOS technology. The HMOS is four times faster than NMOS.

- After 1980, fourth generation µp known as 32-bit, 64 bit microprocessor are launched.

Application of Microprocessor :-

1. Microprocessor Based Dedicated Controllers

→ The dedicated controller is designed to control some physical parameters like temperature, pressure, level, displacement etc.

→ This system is preprogrammed system i.e user cannot change the program. In this system, the user can set only parameters, not program. The program is stored in non-volatile memory like ROM.

eg Motor speed controller, elevator controller, traffic-light controller, security & fire alarms

Home appliances like water heater, microwave oven, video games

Dr. Anuja Bhargava

Anuja

2. Work Stations

→ It is usually 16 or 32 bit µp based system.

eg 3D Graphics

3. Communication System

→ µp can be used in data communication.

eg Serial to parallel and parallel to serial data transfer signal processing.

4. Automobile

→ µp is also used in automobile.

eg Ignition control, Control panel.

5. Medical

→ µp is used in many medical instruments.

eg X - Ray Machine, ECG Machine, pathological labs etc.

6. Instruments

→ µp is used in testing and measuring instruments to control all the functions.

eg CRO, Signal Generator, Power Supply, Touch Screen

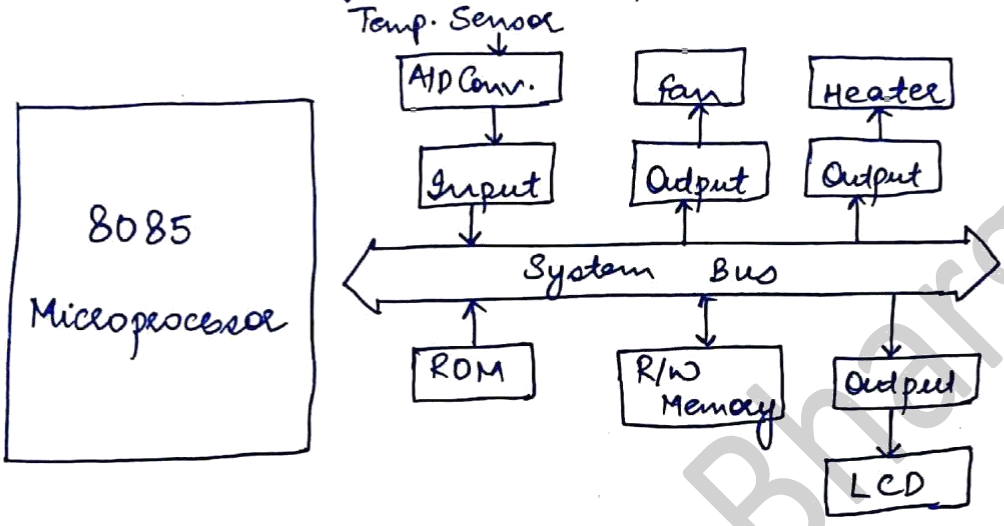
7. Replacement of Random logic

→ Any random logic circuit like combinational and sequential can be replaced by the µp.

Dr. Anuja Bhargava

Application : Microprocessor Controlled Temperature System

- This system is expected to read the temperature in a room, display the temperature at liquid crystal display (LCD) panel, turn on the fan if the temperature is above a set point, and turn on heater if the temperature is below a set point.



- To design such system,
  - → one input device (to sense room temperature)
  - → three output device (Fan, Heater, LCD display)

- Microprocessor

  It will read the binary instructions from memory and execute those instructions continously.

  It will read the temperature, display it at the LCD display panel and turn ON/OFF the fan and heater based on the temperature.

Dr. Anuja Bhargava

- Memory

The system includes two types of Memory.

ROM (Read Only Memory) will be used to store the programs, that is responsible for providing the necessary instructions to the processor to Monitor the system.

The R/W Memory is needed for temporary storage of data.

- Input

In this system, a device is required that translate temperature into equivallent electrical signal. A temperature sensor is a three terminal semiconductor device that generates a voltage signal proportional to the temperature. Since, it is an analog signal. Therefore Analog to Digital convertor is used to convert signal into digital bits.

- Output

FAN - It is turned ON by the processor when the temperature reaches at high limit.

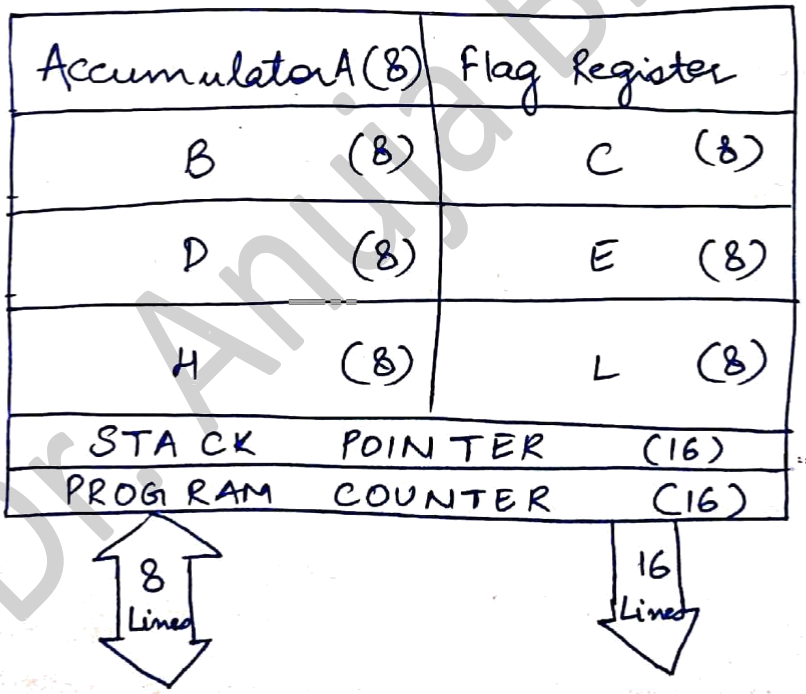
HEATER - It is turned ON by the processor when the temperature reaches at lower limit.

LCD - Liquid Crystal Display can display letters, decimal digits, or graphic characters.

Dr. Anuja Bhargava

8085 Programming Model

- Programming Model (Information needed to write programs)
- The programming model does not reflect the physical structure of 8085 but includes the information that is critical in writing assembly language programs.
- The model includes six registers, one accumulator and one flag register. It has two 16 bit registers: stack pointer and program counter.



Dr. Anuja Bhargava

• Registers

→ The 8085 has six general purpose registers to store 8- bit data identified as, B, C, D, E, H & L.

→ They can be combined as register pairs - BC, DE & HL, to perform some 16 bit operations.

→ The programmer can use these registers to store or copy data into registers.

• Accumulator

→ The accumulator is an 8- bit register that is part of arithmetic / logic unit (ALU).

→ This register is used to store 8- bit data and to perform arithmetic and logical operations.

→ The result of operation is stored in the accumulator.

→ The accumulator is also identified as register A.

• Flags

→ The ALU includes five flip- flops, which are set or reset according to operation.

→ The bit position in flag register is as shown.

D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0
S	Z		AC		P		CY

Dr. Anuja Bhargava

→ Z - Zero : The zero flag is set to one when the result is zero, otherwise it is reset.

→ CY - Carry : If an arithmetic operation results in a carry, the CY flag is set ; otherwise it is reset.

→ S - Sign : The sign flag is set if bit $D_7 = 1$, otherwise it is reset.

→ P - Parity : If the result has even number of 1's, the flag is set ; for an odd number of 1's, the flag is reset.

→ AC - Auxillary Carry : In an arithmetic operation, when carry is generated by digits D_3, and passed to digit D_4, the AC flag is set.

• Program Counter (PC) :

→ The size of this register is 16 bit because the memory address is of 16 bits.

→ The PC points to the memory address from which instruction has to be fetched.

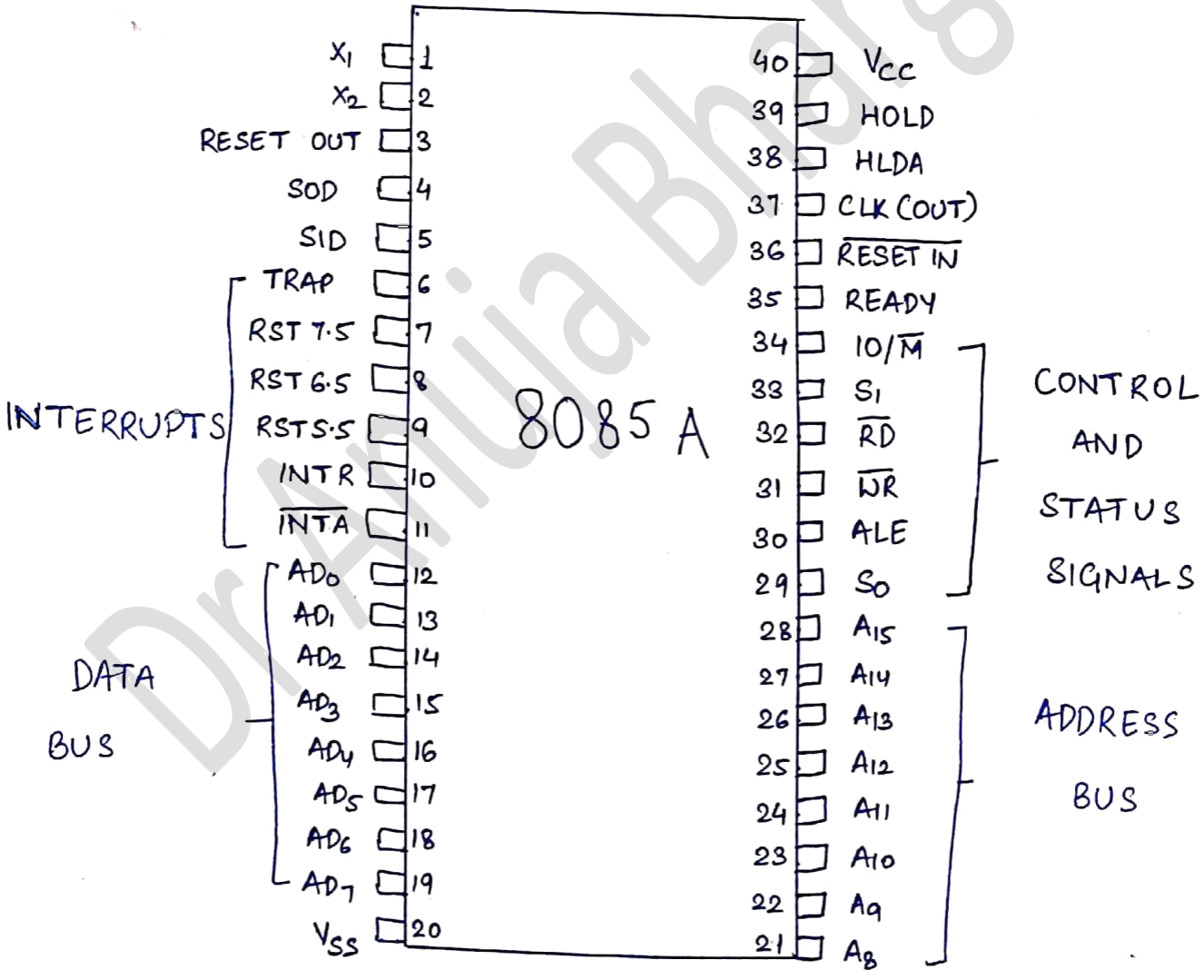
→ The SP points to memory location in R/W memory, called the stack.

Dr. Anuja Bhargava

Anuja

8085 Pin Diagram :-

- The 8085 A (commonly known as 8085) is an 8-bit general purpose microprocessor capable of addressing 64 K of memory.
- The device has 40 pins, require a +5V single power supply, and can operate with 3MHz single-phase clock.
- Fig shows the logic pindiagram of 8085.



Dr Anuja Bhargava

• All the signals can be classified into six groups.

1) ADDRESS BUS :

→ The 8085 has signal lines (pins) that are used as address bus : however, these lines are split into two segments : $A_{15} - A_8$ and $AD_7 - AD_0$.

→ The eight signal lines $A_{15} - A_8$ are unidirectional. These lines are also known as higher - order address.

2) DATA BUS :

→ The signal lines $AD_7 - AD_0$ are bidirectional. These are used as lower order address bus and data bus.

3) CONTROL AND STATUS SIGNALS :-

→ This include following signal

a) ALE - Address latch Enable

→ If ALE = 1, i.e enable, it indicates that the bits on $AD_7 - AD_0$ are address bits.

→ If ALE = 0 ; i.e disable, it indicates that the bits on $AD_7 - AD_0$ are data.

b) $\overline{RD}$ (Read)

→ This is active low signal.

→ This signal indicates that the selected I/O or memory device is to be read and data is available on data bus.

Dr Anuja Bhargava

c) $\overline{WR}$ (Write)

→ This is active low signal.

→ This signal indicates that the data on data bus are to be written into memory or I/O.

d) IO / $\overline{M}$

→ When it is high, it indicates Input - Output operation.

→ When it is low, it indicates a memory operation.

e) S_1 & S_0

→ These status signals, can identify various operations but they are rarely used in small systems

4) POWER SUPPLY & CLOCK FREQUENCY

→ V_{CC} : +5V power supply

→ V_{SS} : Ground Reference

→ X_1, X_2 : A crystal oscillator (RL or RC) is connected at these two pins. The frequency is internally divided by two, therefore to operate at 3MHz, the crystal should have 6MHz frequency.

→ CLK (OUT) : Clock Output used as clock for other devices.

Dr Anuja Bhargava

5) EXTERNALLY INITIATED SIGNALS

→ INTR (Input) Interrupt Request

It is used as general purpose ~~register~~ interrupt.

→ $\overline{\text{INTA}}$ (Output) Interrupt Acknowledge

It is used to acknowledge the interrupt.

→ RST 7.5, 6.5, 5.5

These interrupts transfer the program control to specific memory location.

These have high priorities compare to INTR. interrupt.

Among these the priority order is 7.5, 6.5, 5.5.

→ TRAP (Input)

This is nonmaskable interrupt and has highest priority.

→ HOLD (Input)

This signal indicates that peripheral device such as DMA (direct memory access) is requesting the use of address and data bus.

→ HLDA (Output) Hold Acknowledge

This signal acknowledges the HOLD request.

→ READY

This signal is used to delay the microprocessor read or write cycles.

Dr Anuja Bhargava

→ $\overline{\text{RESET IN}}$

When this signal is low, the program counter is set to zero.

→ RESET OUT

This signal can be used to reset other devices.

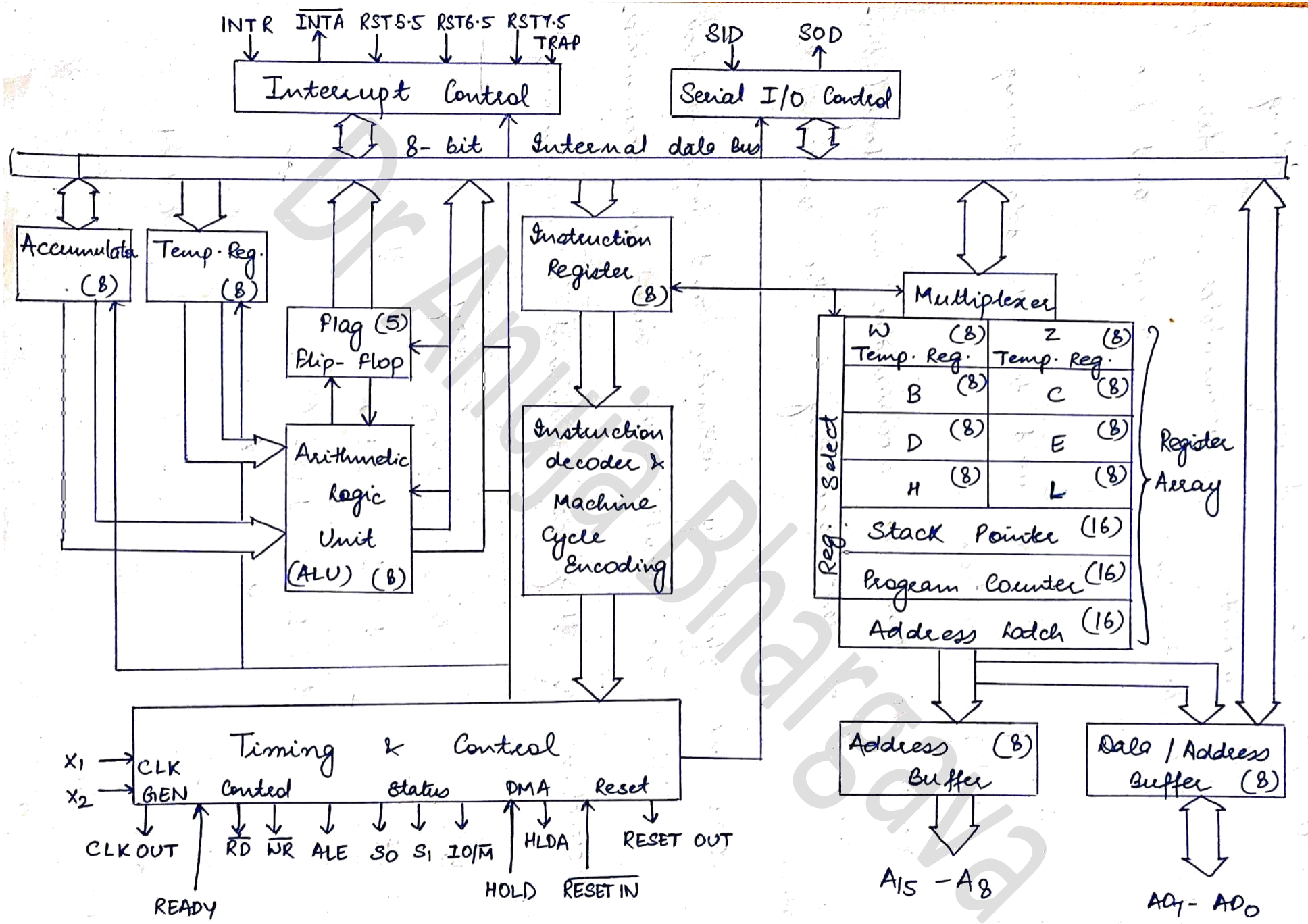
6) SERIAL I/O PORTS

SID (Serial Input Data)

SOD (Serial Output Data)

In serial transmission, data bits are sent over a single line, one bit at a time.

Dr Anuja Bhargava



Dr Anita Bharadwaj

• Fig shows the internal architecture of 8085.

(i) ALU

→ The arithmetic / logic unit perform the computing functions ; it includes the accumulator, the temporary register, the arithmetic & logic circuit and five flags.

→ The temporary register is used to hold data during arithmetic / logic operation.

→ The results are stored in accumulator and flags (flip-flops) are set or reset according to the result of the operation.

→ The flags are affected by arithmetic and logic operation in the ALU. The results are stored in accumulator.

The description of flags are mentioned in programming model.

eg BC

$$\begin{array}{r} +\ DE \\ \hline 9A \end{array}$$

$$\begin{array}{r} 1\ 0\ 1\ 1 \quad 1\ 1\ 0\ 0 \\ +\ 1\ 1\ 0\ 1 \quad 1\ 1\ 1\ 0 \\ \hline 1\ 1\ 0\ 0\ 1 \quad 1\ 0\ 1\ 0 \\ 9 \qquad\quad A \end{array}$$

1 1 1 1 ← Carry

If MSB = 1, S = 1

Z = 0 ∵ ALB = X

AC = 0

P = 1

CY = 1

12 + 14 = 26

26 - 16 = 10

11 + 13 = 24

25 - 16 = 9

S	Z		AC		P		CY

Dr Anuja Bhargava

(ii) Timing and Control Unit

→ This unit synchronizes all the microprocessor operations with the clock and generates the control signal necessary for communication between the microprocessor and peripherals.

(iii) Instruction Register and Decoder.

→ It is the part of ALU.

→ When an instruction is fetch from memory, it is loaded to the instruction register.

→ The decoder decodes the instruction and establishes the sequence of event to follow.

→ The instruction register is not programmable and cannot be accessed through any instruction.

(iv) Register Array

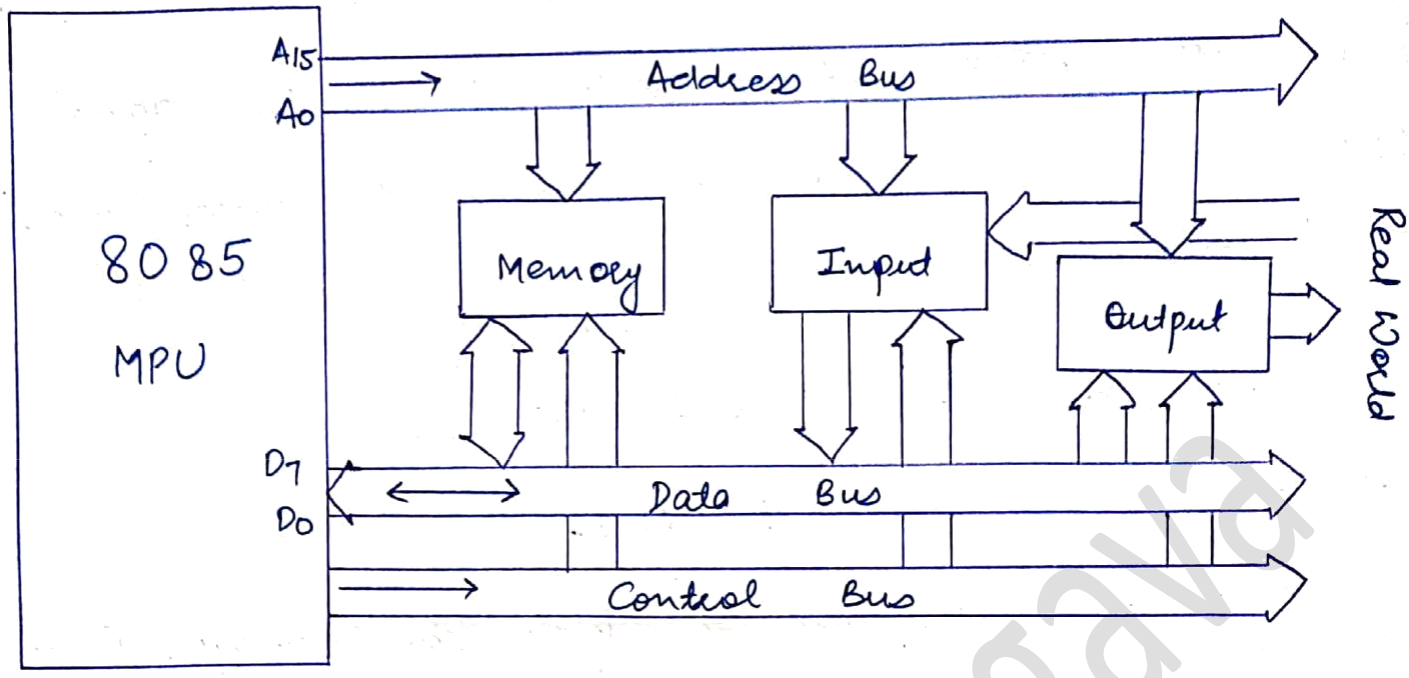
→ Two registers (temporary registers) W & Z are included in the register array.

→ These registers are used to hold 8-bit data during during the execution of some instructions.

→ As these are used internally, they are not available to the programmer.

Dr Anuja Bhargava

Microprocessor Bus Structure



Address Bus

→ It is a group of 16 lines generally identified as $A_0 - A_{15}$.

→ It is unidirectional, bit flows in one direction i.e. from MPU to peripheral devices.

→ The MPU uses address bus to perform the function: Identify peripheral device or Memory location.

Data Bus

→ It is a group of eight lines for data flow.

→ These lines are bidirectional i.e data flows in both direction between MPU & Memory & peripheral devices.

→ The MPU use data bus to perform: transferring binary information

Dr Anuja Bhargava

Note :- → The 8085 MPU with its 16 address lines is capable of addressing $2^{16} = 65536$ (generally known as 64 K) of memory location.

→ The 8 bit Data ranging from 00 to FF ($2^8 = 256$)

The largest number that appear on data bus is

1 1 1 1 1 1 1 1 (255_{10}).

Control Bus

→ It provide timing signal.

→ It compose of various single lines that carry synchronization signals.

Therefore, to communicate with peripheral or memory, the MPU needs to perform the following steps

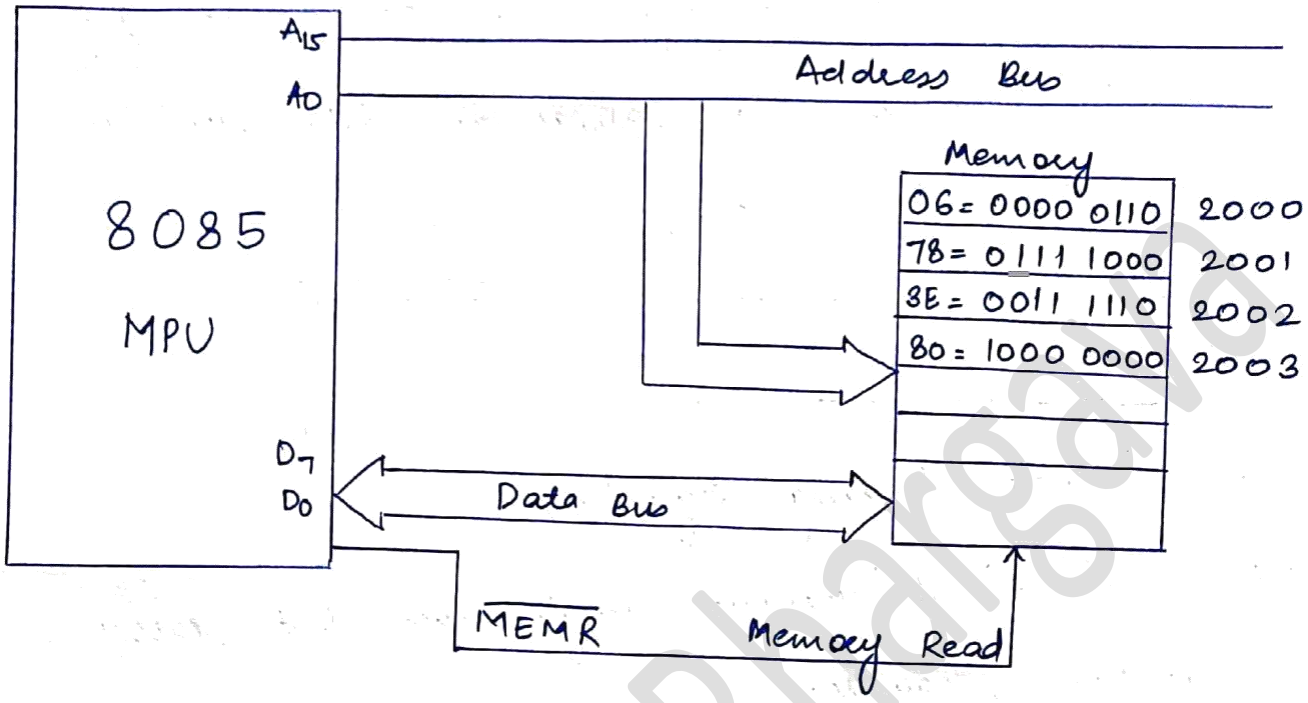
Step 1) Identify the peripheral or memory location. (Address Bus)

Step 2) Transfer binary information. (data bus)

Step 3) Provide timing or synchronization signals. (control Bus)

Dr Anuja Bhargava

Example: To communicate with memory, to read an instruction from memory location - the MPU places the 16-bit address on the address bus.



- The address on bus is decoded by external logic circuit.
- The MPU sends a pulse called Memory Read as the control signal.
- The pulse activates the memory chip, and the content of memory location are placed on the data bus and brought inside the microprocessor.

Dr Anuja Bhargava

Instruction :-

- An instruction is a command given in binary pattern to the microprocessor to perform a specified operation on given data.
- The entire group of instructions is called as Instruction set of microprocessor.
- Each instruction consist of two parts : opcode & operand.

→ Opcode

The first part of instruction which specifies the operation to be performed by the computer is called opcode.

→ Operand

The second part of the instruction is the data to be operated on, is called operand.

- The various techniques to specify data or operand for instruction is given below

a) 8-bit or 16-bit data may be directly given in the instruction itself.

eg MVI A, 02 H — 02H is the 8-bit data.

LXI H, 1500 H — 1500H is the 16-bit data.

H indicates hexadecimal data.

Dr Anuja Bhargava

b) The address of Memory location or I/O port address may be given in instruction itself.

eg STA 5000H 5000H is the 16-bit memory address.

c) In some instruction only one register is specified. The content of the specified register is one of the operand

eg ADD B The content of B register is one operand and the content of A register is other operand

d) Some instruction specify one or two register. The content of registers are the required data.

eg MOV A, B Two registers are specified i.e A & B.

INR B One register is specified i.e B.

e) In some ~~register~~ instruction data is not specified. The most instruction of this type operate on the content of accumulator.

eg CMA Data is in A register.

Dr Anuja Bhargava

Instruction format

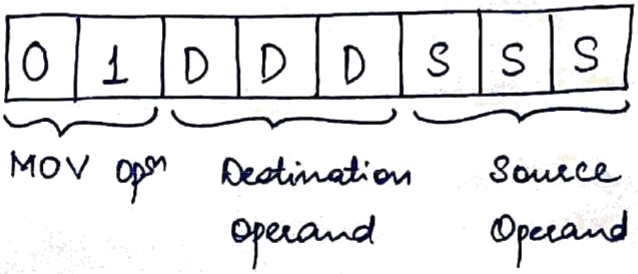
- A digital computer / microprocessor understood instruction in binary codes known as opcodes.
- Due to different ways of specifying data or operand for instructions, the machine code of all instructions are not of same length.
- The size of machine code is one, two or three byte known as Instruction Word Size.
- Depending upon the size of machine codes, the 8085 instructions are classified into three types

(1) One Byte Instruction

A 1 byte instruction include the opcode and operand in the 8-bit only ie one byte.

format | Opcode |

eg (i) MOV Instruction

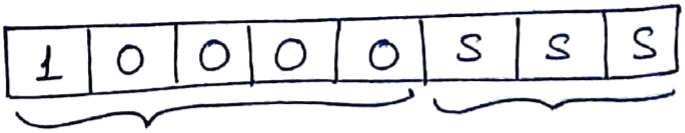


MOV A, B = 01 111 000

= 78 H

Dr Anuja Bhargava

(ii) ADD Instruction



ADD B = 10000 000 = B0 H

(2) Two - Byte Instruction

In two byte instruction the first byte is opcode and second byte is operand.

format

Opcode	Operand

eg (i) MVI B, 05 H Move 05 to register B.

06 , 05

The first byte 06 is the opcode for MVI B and the second byte is operand.

(ii) IN 02 Read the data at port

DB 02

The first byte DB is the opcode for IN and the second byte is operand i.e 8- bit address of port from which the data is transfered to A register.

Dr Anuja Bhargava

(3) Three - Byte Instruction

In three byte instruction the first byte is opcode and second and third bytes are operands.

Format:

Opcode	Operand 1	Operand 2

eg (i) LXI H, 2500 H Load HL pair with 2500 H.

21, 00, 25 Code form

The first byte 21 H is the opcode for instruction LXI H. The second byte 00 is the lower byte of data, which is loaded in register L. The third byte 25 is the higher byte of data loaded in register H.

(ii) STA 2600 H Store the content of A register to memory location 2600 H.

32, 00, 60 Code form.

The first byte 32 is the opcode for instruction STA. The second byte 00 is lower byte of address of memory location 2600 H & The third byte 26 is the higher byte of the address of memory location 2600 H.

Dr Anuja Bhargava

Opcode Format:

- Each instruction has a unique code.
- The opcode contains information regarding operation, register used, memory to be used.
- The opcode for each instruction is fixed.

Register	Code
B	000
C	001
D	010
E	011
H	100
L	101
M	110
A	111

Register Pair	Code
BC	00
DE	01
HL	10
Sf or SB	11

eg (1) MOV r_1, r_2 Transfer content of r_2 register to r_1 register.

Opcode

0	1	D	D	D	S	S	S

MOV A, B 01 111 000 = 78H

Dr Anuja Bhargava

MOV A, C 01 111 001 = 79 H

MOV A, M 01 111 110 = 7E H

(ii) LXI rp, data load HL pair with 16-bit data.

Opcode

0	0	D	D	0	0	0	1

LXI B, Data 00 00 0001 = 01 H

LXI H, Data 00 10 0001 = 21 H

LXI D, Data 00 01 0001 = 11 H

(iii) ADD r Add register to accumulator.

Opcode

1	0	0	0	0	S	S	S

ADD B 10000 000 = 80 H

ADD C 10000 001 = 81 H

ADD E 10000 011 = 83 H

Dr Anuja Bhargava

Addressing Modes :-

- The various ways of specifying data (or operands) for instructions are called as addressing modes.
- The 8085 addressing modes are classified into following types

(i) Register Addressing Mode

→ In register addressing mode the source and destination operation are in the general purpose registers.

→ The opcode specifies the operation and registers to be used to perform the operation.

→ eg MOV A, B Move the content of register B to register A.

ADD C Add the content of register C to register A. Store result in A.

(ii) Direct Addressing Mode

→ In direct addressing mode, the address of the operand is given in the instruction itself.

→ eg STA 7500H Store the content of accumulator in memory location 7500 H.

Dr Anuja Bhargava

→ IN, 01 Read data from port 01 or B.

(iii) Indirect Addressing Mode

→ In indirect addressing mode the address of operand is specified by register pair.

→ The address stored in register pair points to memory location.

→ eg LXI H, 7500H Load HL with 7500 H.

MOV A, M Move the content of memory location whose address is in HL pair, to the accumulator.

LXI H, 4000 H Load HL pair with 4000 H.

ADD M Add the content of memory location, whose address is in HL pair, to the content of accumulator.

(iv) Immediate Addressing Mode

In immediate addressing mode the data or operand is specified with in the instruction itself or it is a part of instruction.

Dr Anuja Bhargava

eg MVI A, 05H Move 05H in register A.

LXI, H, 2600H Load HL pair with 2600H i.e 16-bit data.

(V) Implicit or Implied Addressing Mode

→ In implicit addressing mode, the operand is not specified in the instruction.

→ The operand is specified with in the opcode itself.

→ The data is supposed to be present generally in accumulator.

eg RAL Rotate the content of the accumulator towards left.

(Whenever RAL is used it is implied that the data to be operate on is available in the instruction itself.)

Dr Anuja Bhargava

8085 Instruction Set :-

The 8085 instructions are classified into six groups

1. Data Transfer Group
2. Arithmetic Group
3. Logical Group
4. Branch Control Group
5. Stack, I/O and Machine Control Group
6. Interrupt Control Group.

(1) Data Transfer Group:

→ The instruction of this group transfer data from source to destination without changing the content of source.

→ The flag content is not affected by any instruction.

→ The instruction of this group perform various transfer as:

Data Transfer Group

Register to Register	Register to Memory	Memory to Register	Data to Register	Data to Memory
MOV r1, r2	MOV M, r	MOV r, M	MVI r, data	MVI M, data
XCHG	STA addr	LDA addr	LXI rp, data	
	SHLD addr	LHLD addr		
	STAX rp	LDAX rp		

Dr Anuja Bhargava

S.NO.	Instruction	Description	No. of Bytes	Addressing Mode	Example	
					Before Execution	After Execution
1.	MOV r_1, r_2 $[r_1] \leftarrow [r_2]$	The content of register r_2 is moved to register r_1	1	Register	MOV A, B [A]= 25H [B] = 63H	 [A]= 63H [B] = 63H
2.	MOV r, M $[r] \leftarrow [[HL]]$	The content of memory whose address is in register pair rp, moved to register r.	1	Indirect	MOV A, M HL = 2500H [2500] = 75H [A] = 33H	 HL = 2500H [2500] = 75H [A] = 75H
3.	MOV M, r $[[HL]] \leftarrow r$	The content of register r is moved to memory location address by HL pair	1	Indirect	MOV M, A HL = 2500H [2500] = 75H [A] = 33H	 HL = 2500H [2500] = 33H [A] = 33H
4.	MVI r, data $[r] \leftarrow$ data	Data is moved to register r.	2	Immediate	MVI B, 53H [B] = 38H	 [B] = 53H
5.	MVI M, data $[[HL]] \leftarrow$ data	The data is moved to memory location whose address is in HL pair.	2	Immediate / Indirect	MVI M, 14H HL = 2500H [2500] = 73H	 HL = 2500H [2500] = 14H
6.	LXI rp, data 16	The 16 bit data is moved to register pair rp.	3	Immediate	LXI H, 1234H [HL] = 5678 [H] = 56 [L] = 78	 [HL] = 1234 [H] = 12 [L] = 34
7.	LDA addr [A] $\leftarrow$ addr	The content of memory location whose address is specified in instruction, is loaded into accumulator.	3	Direct	LDA 7000H [A]= 55H [7000] = 11H	 [A] = 11H [7000] = 11H

Dr Anuja Bhargava

8.	STA addr [addr] ← [A]	The content of accumulator is stored in memory location whose address is specified	3	Direct	STA 7000H [A] = 55H [7000] = 11H	STA 7000H [A] = 55H [7000] = 55H
9.	LHLD addr L ← addr H ← addr+1	The content of memory location whose address is specified is loaded.	3	Direct	LHLD 7000 [7000] = 34H [7001] = 12H [L] = 55H [H] = 44H	LHLD 7000 [7000] = 34H [7001] = 12H [L] = 34H [H] = 12H
10.	SHLD addr addr ← L addr+H ← H	The content of register is stored at specified memory location.	3	Direct	SHLD 7000 [7000] = 34H [7001] = 12H [L] = 55H [H] = 44H	SHLD 7000 [7000] = 55H [7001] = 44H [L] = 55H [H] = 44H
11.	LDAX rp [A] ← [[rp]]	The content of memory location, whose address in rp, is loaded in acc.	1	Indirect	LDAX D [DE] = 5000H [5000] = 11H [A] = 22H	LDAX D [DE] = 5000H [5000] = 11H [A] = 11H
12.	STAX rp [[rp]] ← [A]	The content of accumulator is stored in memory location whose address is in rp	1	Indirect	STAX B [BC] = 7500H [7500] = 11H [A] = 22H	STAX B [BC] = 7500H [7500] = 11H [A] = 22H
13.	XCHG	Exchange the content of HL with DE pair.	1	Register	[HL] = 1234H [DE] = 5678H	[HL] = 5678H [DE] = 1234H

Dr Anuja Bhargava

(2) Arithmetic Group

- The instruction of this group perform arithmetic operation such as addition, subtraction, increment or decrement of the content of register or memory.
- The instruction of this group perform various arithmetic operation such as

Dr Anuja Bhargava

Arithmetic Group

Addition	Subtraction	Increment	Decrement
8-bit	8-bit	8-bit	8-bit
ADD r	SUB r	INR r	DCR r
ADD M	SUB M	INR M	DCR M
ADI data	SUI data		
8-bit with carry	8-bit with borrow	16-bit	16-bit
ADC r	SBB r	INX rp	DCX rp
ADC M	SBB M		
ACI data	SBI data		
8-bit decimal			
DAA			
16-bit			
DAD rp			

- All flags are affected.

S.No.	Instruction	Description	No. of Bytes	Addressing Mode	Example Before Execution	Example After Execution
1.	ADD r [A] ← [A] + [r]	The content of register is added to the content of accumulator & stored in accumulator.	1	Register	ADD B [A] = 35H [B] = 77H	[A] = ACH [B] = 77H
2.	ADD M [A] = [A] + [[HL]]	The content of memory location addressed by HL pair is added to the content of ammulator. & stored in accumulator.	1	Indirect	HL = 2500H [2500] = 73 [A] = 57	HL = 2500H [2500] = 73 [A] = CA H

Dr Anuja Bhargava

3.	ADI data [A] ← [A] + data	The data is added to content of accumulator. The result is stored in accumulator.	2	Immediate	ADI 67 [A] = 59H [A] = 59+67 = C0 H
4.	ADC r [A] ← [A] + [r] + [CY]	The content of register & carry flag status are added to the content of accumulator & result is stored in accumulator	1	Register	ADC B [A] = 57H [B] = 63H [CY] = 1 [A] = 57+63 H = BB H [B] = 63 H CY = 0
5.	ADC M [A] ← [A] + [[HL]] + [CY]	The content of memory location addressed by HL pair & CY flag are added to accumulator.	1	Indirect	ADC M [A] = AB H [CY] = 1 HL = 2500H [2500] = CDH [A] = AB+CD+1 = 79 H HL = 2500 H [2500] = CD H
6.	ACI data [A] ← [A] + data + [CY]	The data & carry flag are added to the accumulator.	2	Immediate	ACI 77 [A] = A2 H [CY] = 1 [A] = A2+77+1 = 1B [CY] = 1
7.	DAD rp [HL] ← [HL] + [rp]	The content of register pair are added to the content of HL pair.	1	Register	DAD D [HL] = ABC9 [DE] = 7890 [HL] = 2459H
8.	SUB r [A] ← [A] - [r]	The content of register r is subtracted from content of accumulator.	1	Register	SUB B [A] = 12 H [B] = 34 H [A] = DE H [B] = 34 H
9.	SUB M [A] ← [A] - [[HL]]	The content of memory location addressed by HL pair is subtracted from content of accumulator.	1	Indirect	SUB M [HL] = 4500H [4500] = 78H [A] = AB H [HL] = 4500H [4500] = 78H [A] = AB-78 = 33 H
0.	SUI data [A] ← [A] - data	The data is subtracted from the content of accumulator.	2	Immediate	SUI 77 [A] = FE H [A] = FE-77 = 87 H

Dr Anuja Bhargava

11.	SBB r [A] ← [A] - [r] - [CY]	The content of register & carry flag are subtracted from the content of accumulator.	1	Register	SBB C [A] = 78H [C] = 39H CY = 1	[A] = 38H [C] = 39H CY = 1
12.	SBB M [A] ← [A] - [[HL]] - [CY]	The content of memory location by HL pair and carry flag are subtracted from the content of accumulator.	1	Indirect	SBB M [HL] = 7500H [7500] = 67H [A] = 34H [CY] = 1	[HL] = 7500H [7500] = 67H [A] = 34 - 67 - 1 = CCH
13.	SBI data [A] ← [A] - data - [CY]	The data & carry flag is subtracted from the content of accumulator.	2	Immediate	SBI 77 [A] = ABH [CY] = 0	[A] = AB - 77 = 34H
14.	INR r [r] ← [r] + 1	The content of register is incremented by one.	1	Register	INR B [B] = FFH	= 00H [B] = FF + 1
15.	INR M [HL] ← [HL] + 1	The content of memory location addressed by HL pair is ↑ment by 1	1	Indirect	INR M [HL] = 7500H [7500] = 78H	[HL] = 7500H [7500] = 79H
16.	INX rp [rp] ← [rp] + 1	The content of register pair rp is incremented by one.	1	Register	INX H [HL] = FFFFH	[HL] = FFFF + 1 = 0000H
17.	DCR r [r] ← [r] - 1	The content of register r is decremented by one.	1	Register	DCR B [B] = 00H	[B] = 00 - 1 = FFH
18.	DCR M [HL] ← [HL] - 1	The content of memory location addressed by HL pair is decrement by one.	1	Indirect	DCR M [HL] = 7500H [7500] = 79H	[HL] = 7500H [7500] = 78H
19.	DCX rp [rp] ← [rp] - 1	The content of register pair rp is decremented by one.	1	Register	BC = 0000H	BC = 0000 - 1 = FFFFH
20.	DAA Decimal Adjust Accumulator	It converts the Hex result in accumulator into decimal or BCD. It is used after ADD, ADI, ACI, ADC. → If value of 4 LSB of A79 or AC=1, 06 is added to accumulator. → If the value of 4 MSB of A79 or CY=1 60 is added to accumulator.	1	Implicit	ADD B DAA [A] = 27H [B] = 46H	[A] = 27 + 46 = 6D + 06 = 73H [B] = 46H

Dr Anuja Bhargava

(3) Logical Group :-

- The instruction of this group perform logical operation such as AND, OR, EX-OR, compare, rotate or implement of data in registee or memory.
- Most of the instruction affect all flags.

Logical Group

AND	OR	EX-OR	Complement	Set	Compare	Rotate
ANA r	ORA r	XRA r	CMA	STC	CMP r	RLC
ANA M	ORA M	XRA M	CMC		CMP M	RRC
ANI data	ORI data	XRI data			CMP data	RAL
						RAR

S.NO.	Instruction	Description	No. of B.	Addressing Mode	Before Execution	After Execution
1.	ANA r [A] ← [A] ∧ r	The content of r is logically ANDed with the content of acc. The result is stored in accumulator.	1	Register	ANA B [A] = 78 H [B] = 0B H	[A] = 08 H [B] = 0B H
2.	ANA M [A] ← [A] ∧ [HL]	The content of memory location addressed by HL pair is logically ANDed with the content of accumulator. The result is stored in accumulator.	1	Indirect	ANA M HL = 7500 H [7500] = 0F H [A] = [89]	HL = 7500 H [7500] = 0F H [A] = 09 H
3.	ANI data [A] ← [A] ∧ Data	The data is logically ANDed with the content of accumulator. The result is stored in accumulator.	2	Immediate	ANA F0 [A] = 89 H	[A] = 80 H
4.	ORA r [A] ← [A] ∨ r	The content of r is logically ORed with the content of acc. The result is stored in acc.	1	Register	ORA B [A] = 78 H [B] = 56 H	[A] = 7E H [B] = 56 H
5.	ORA M [A] ← [A] ∨ [HL]	The content of memory location addressed by HL pair is logically ORED with the content of accumulator	1	Indirect	ORA M HL = 7500 H [7500] = 55 H [A] = AA H	HL = 7500 H [7500] = 55 H [A] = FF H

Dr Anuja Bhargava

6.	ORI data [A]←[A] V data	The data is logically ORed with the content of accumulator. The result is stored in accumulator.	2	Immediate	ORI 77 [A] = 00H	[A] = 77H
7.	XRA r [A]←[A] ⊕ r	The content of register r is logically EX-ORed with the content of acc. The result is stored in accumulator.	1	Register	XRA B [A] = 55H [B] = AAH	[A] = FFH [B] = AAH
8.	XRA M	The content of memory location add. by HL pair is logically EX-ORed with the content of accumulator. The result is stored in accumulator.	1	Indirect	XRA M HL = 7500H [7500] = 55H [A] = 55H	HL = 7500H [7500] = 55H [A] = 00H
9.	XRI data [A]←[A] ⊕ data	The data is logically EX-ORed with the content of accumulator. The result is stored in accumulator.	2	Immediate	XRI 78 [A] = 00H	[A] = 78H
10.	CMA	1's complement of the content of the accumulator is obtained and the result is stored in accumulator.	1	Implicit	CMA [A] = 55H	[A] = AAH
11.	CMC [CY]←[$\overline{CY}$]	Complement Carry status The CY flag is complemented.	1	Implicit	CMC [CY] = 1	[CY] = 0
12.	STC CY ← 1	Set Carry Flag The carry flag is set to 1.	1	Implicit	STC [CY] = 0	[CY] = 1
13.	CMP r [A] - [r] A > r, CY = 0 A < r, CY = 1 A = r, Z = 1	The content of register r is subtracted from content of accumulator. The result is not stored anywhere.	1	Register	CMP B [A] = 77H [B] = 99H	[A] = 77H [B] = 99H CY = 1
14.	CMP M [A] - [[HL]] A > M, CY = 0 A < M, CY = 1 A = M, Z = 1	The content of memory location addressed by HL pair is subtracted from the content of accumulator. The result is not stored anywhere.	1	Indirect	CMP M HL = 5000H [5000] = 32H [A] = 55H	HL = 5000H [5000] = 32H [A] = 55H [CY] = 0

Dr Anuja Bhargava

15. CPI data — The data is subtracted from the content of accumulator. The result is not stored anywhere. — 2 — Immediate — CPI 57

[A] - data
A > data, CY=0
A < data, CY=1
A = data, Z=1

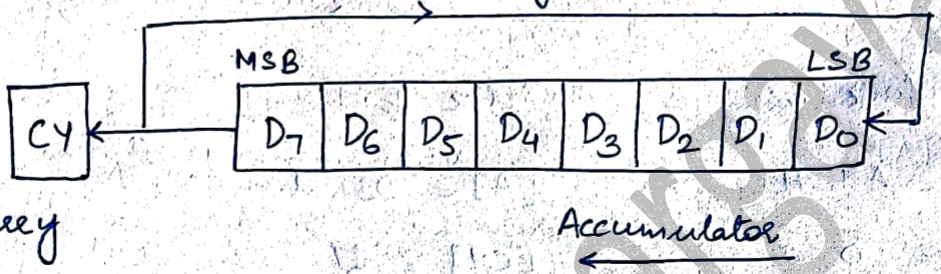
[A]=57H [A]=57H
Z=1

16. RLC — (Rotate Accumulator left) — 1 — Implicit — RLC

$D_{n+1} \leftarrow D_n$
$D_0 \leftarrow D_7$
$CY \leftarrow D_7$

The content of accumulator is rotated towards left by one bit.

[A]=A7H [A]=4EH
[CY]=0 [CY]=1

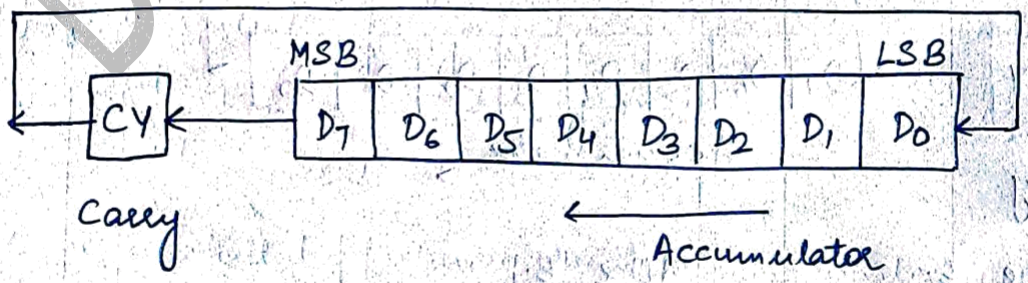


eg 0 1 0 1 0 0 1 1 1 — RLC → 1 0 1 0 0 1 1 1
0 A7 H — 1 4 F H

17. RAL — (Rotate Accumulator left through Carry) — 1 — Implicit

$D_{n+1} \leftarrow D_n$
$CY \leftarrow D_7$
$D_0 \leftarrow CY$

The content of the accumulator is rotated towards left through carry. The seventh bit of accumulator is moved to carry and the carry bit is move to zero bit of accumulator.



Before E.
[A] = A7 H
[CY] = 0

After E.
[A] = 4E H
[CY] = 1

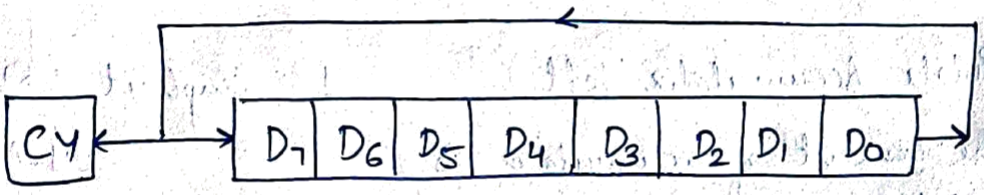
0 1 0 1 0 0 1 1 — RAL → 1 0 1 0 0 1 1 1 0
A7 H — 4E H

Dr. Anita Bhargava

18. RRC (Rotate Accumulator Right) 1 Implicit

$[D_{n-1}] \leftarrow [D_n]$
$[D_7] \leftarrow [D_0]$
$[CY] \leftarrow [D_0]$

The content of accumulator is rotated towards right by one bit. The zero bit of the acc. is moved to the seventh bit of the acc. as well as to carry bit.



Carry Accumulator →

Before E.
[A] = A7 H
[CY] = 0

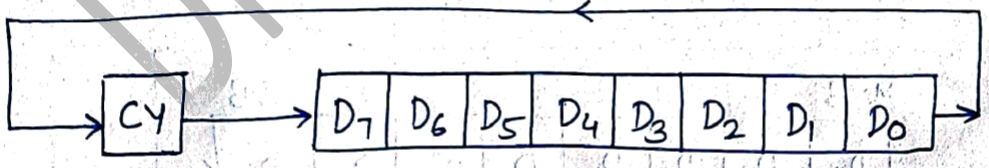
After E.
[A] = D3 H
[CY] = 1

0 1010 0111 —RRC→ 1 1101 0011

19. RAR (Rotate Accumulator Right through Carry) 1 Implicit

$[D_{n-1}] \leftarrow [D_n]$
$[CY] \leftarrow [D_0]$
$[D_7] \leftarrow CY$

The content of accumulator is rotated towards right by one bit. The zero bit of the acc. is moved to carry, and the carry bit is moved to the seventh bit of accumulator.



Carry

Before E.
[A] = A7 H
[CY] = 0

After E.
[A] = 53 H
[CY] = 1

0 1010 0111 —RAR→ 1 0101 0011

Dr Anuja Bhargava

④ Branch Control Group :-

- The instruction of this group change the normal sequence of the program.
- There are two types of branch instruction

(i) Unconditional Branch Instruction :-

It transfer the program to the specified label or address unconditionally i.e without satisfying any condition.

(ii) Conditional Branch Instruction :-

It transfer the program to the specified label or address when certain condition is specified or satisfied. The condition depends upon the status of Zero, Parity, Sign, Carry Flags.

1. JMP addr (label)

→ Unconditional Jump

→ Jump to the address of the label unconditionally. The address of the label is the address of memory location for next instruction to be executed. The instruction load the address of label in PC. (Immediate)

eg JMP 7000

Before Execution	After Execution
PC = 5000 H	PC = 7000 H

This instruction will load the PC with 7000 H and the processor fetch or execute the instruction from this address.

Dr Anuja Bhargava

Conditional Jump Instructions :-

If the condition is true or satisfied the only jump is made at specified address.

If the condition is false or not satisfied then the next instruction in the sequence is executed.

2) JZ add (label) : Jump if the result is zero i.e Z=1.

3) JNZ add (label) : Jump if the result is not zero ie Z=0.

4) JC adde (label) : Jump if there is carry i.e CY =1

5) JNC adde (label) : Jump if there is no carry i.e CY=0

6) JP adde (label) : Jump if the result is plus i.e S=0

7) JM adde (label) : Jump if the result is Minus ie S=1

8) JPE adde (label) : Jump if even parity i.e P=1

9) JPO adde (label) : Jump if odd parity i.e P=0

If condition is true PC ← adde (label)

else PC ← PC + 3

(execute next instruction in sequence)

(Immediate)

eg JZ 7000H

Before Execution	After Execution
[PC] = 5000H	[PC] = 7000 H
[Z] = 1	[Z] = 1

eg JZ 7000 H

[PC] = 5000H	[PC] = 50003 H
Z = 0	Z = 0

Dr Anuja Bhargava

10) Call addr (label)

→ Unconditional Call

→ It is used to call subroutine.

→ Before calling or jumping to the subroutine, the address of next instruction of the main program is saved in the stack.

→ The content of the stack pointer is decremented by two.

→ Then the program jumps to subroutine starting at address specified by label.

→ Immediate / Indirect

eg CALL 5000

Before E.

[SP] = 7000

[PC] = 6000

After E.

[6FFF] = 60

[6FFE] = 00

[SP] = 6FFE

[PC] = 5000

When the instruction is executed, the program counter contents 6000 will be stored on the stack and microprocessor will load the PC with 5000H and starts executing instruction from 5000 onwards.

Conditional CALL Instruction :-

If condition is true then program calls the specified subroutine.

If condition is false, then the next instruction in the sequence is executed.

Dr Anuja Bhargava

Before call, the address of next instruction of the main program is saved in the stack. The stack pointer is decremented by 2

11) CZ addr (label): Call a subroutine if the result is zero, Z=1.

12) CNZ addr (label): Call a subroutine if the result is not zero, Z=0.

13) CC addr (label): Call a subroutine if there is carry i.e CY=1.

14) CNC addr (label): Call a subroutine if there is no carry i.e CY=0.

15) CP addr (label): Call a subroutine if the result is plus ie S=0.

16) CM addr (label): Call a subroutine if the result is negative, S=1.

17) CPE addr (label): Call a subroutine if even parity i-e P=1.

18) CPO addr (label): Call a subroutine if odd parity i-e P=0.

If Condition is true

$$[[SP]-1] \leftarrow PCH$$

$$[[SP]-2] \leftarrow PCL$$

$$[SP] \leftarrow [SP] - 2$$

$$[PC] \leftarrow \text{addr (label)}$$

else

$$PC \leftarrow PC + 3$$

i-e execute the next instruction in the sequence.

Immediate / Indirect.

Dr Anuja Bhargava

Example:

1) CZ 7000

Before E.	After E.
[SP] = 5000	[4FFF] = 60
[PC] = 6000	[4FFE] = 00
[Z] = 1	[SP] = 4FFE
	[PC] = 7000
	[Z] = 1

The program calls a subroutine from 7000 because condition is true i.e Z = 1.

2) CZ 7000

Before E.	After E.
[SP] = 5000	[SP] = 5000
[PC] = 6000	PC = 6003
[Z] = 0	[Z] = 0

The program does not call a subroutine because condition is false i.e Z = 0.

Dr Anuja Bhargava

19) RET

→ Unconditional Return

→ Return from subroutine.

→ RET instruction is used at the end of subroutine.

→ Before the execution of a subroutine the address of the next instruction of the main program is saved in the stack.

→ The execution of RET instruction brings back the saved address from the stack to the program counter.

→ The content of stack pointer is incremented by 2. Then the program jumps to the next instruction after call in the main program.

→ Indirect.

eg
=)

RET

Before E.	After E.
[SP] = 7000 H	[SP] = 7002 H
[7000] = 00 H	[7000] = 00H
[7001] = 50 H	[7001] = 50H
[PC] = 2000 H	[PC] = 5000 H

So the next instruction will be executed from 5000 instead of 2000.

Dr Anuja Bhargava

Conditional RET Instruction :-

If condition is true, then program returns from the subroutine.

If condition is false, then the next instruction in the sequence is executed.

20) RZ : Return from subroutine, if the result is zero i.e Z = 1.

21) RNZ : Return from subroutine, if the result is not zero, i.e Z = 0.

22) RC : Return from subroutine, if there is carry, i.e CY = 1

23) RNC : Return from subroutine, if there is no carry i.e CY = 0.

24) RPE: Return from subroutine, if even parity i.e P = 1.

25) RPO: Return from subroutine, if odd parity i.e P = 0.

If Condition is true

$$[PCL] \leftarrow [[SP]]$$

$$[PCH] \leftarrow [[SP]] + 1$$

$$[SP] \leftarrow [SP] + 2$$

else

$$PC \leftarrow PC + 1$$

i.e execute the next instruction in the sequence.

eg RZ

Before E.

[SP] = 7000H [Z] = 1

[7000] = 00H

[7001] = 50H

[PC] = 2000H

After E.

[SP] = 7002H [Z] = 1

[7000] = 00H

[7001] = 50H

[PC] = 5000

Dr Anuja Bhargava

The program will return from subroutine to main program and start the execution of instruction from 5000 because condition is true i.e Z=1.

24) RZ

Before E.	After E.
[SP] = 7000 H	[SP] = 7000 H
[7000] = 00 H	[7000] = 00 H
[7001] = 50 H	[7001] = 50 H
[PC] = 2000 H	[PC] = 2001 H (PC+1)
[Z] = 0	[Z] = 0

The program does not return from subroutine because condition is false i.e Z=0.

26) RP : Return from subroutine, if result is plus ; S=0.

27) RM : Return from subroutine, if result is negative ; S=1

28) PCHL

→ The content of HL are transfered to program counter. The content of register H are moved to high - order 8- bits of PC. The content of register L are transfer to low - order 8- bits of register PC.

→ Register

eg PCHL

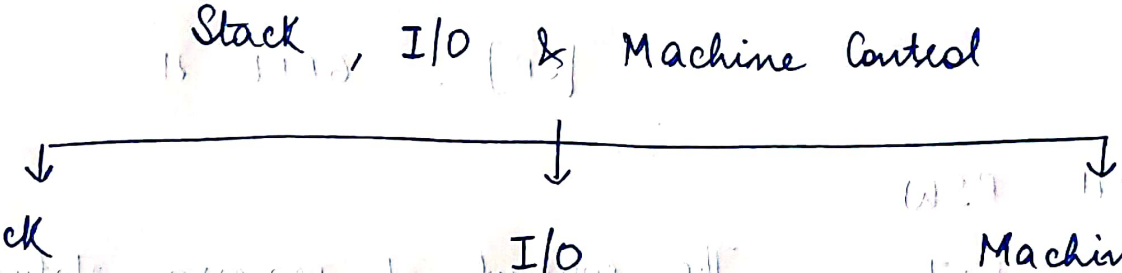
Before Execution	After Execution
[HL] = 7000 H	[HL] = 7000 H
[PC] = 5000 H	[PC] = 7000 H

The program start the execution of instruction

Dr Anuja Bhargava

(5) Stack, I/O and Machine Control Group:-

→ The instructions of this group perform I/O data transfer, manipulates the stack and perform related operations.



Stack	I/O	Machine Control
PUSH rp, PUSH PSW	IN port-address	HLT
POP rp, POP PSW	OUT port-address	NOP
XTHL, SPHL		

1) PUSH rp

$[[SP] - 1] \leftarrow [r_h]$

$[[SP] - 2] \leftarrow [r_l]$

$[SP] \leftarrow [SP] - 2$

The content of register pair rp is pushed on to the stack. The stack pointer is decremented by one & the content of higher order register of rp are copied. The stack pointer again decremented by one and the content of lower order register of rp are added.

No. of Bytes : 1

Addressing Mode: Indirect

Scanned with OKEN Scanner

eg PUSH H

Before Execution

[HL] = 1234 H

[SP] = 7000 H

After Execution

[HL] = 1234 H

[6FFF] = 12 H

[6FFE] = 34 H

[SP] = 6FFE H

2) PUSH PSW

[[SP] - 1] ← [A]

[[SP] - 2] ← Flag or PSW

[SP] ← [SP] - 2

The content of processor status word (PSW) is pushed onto the stack. The stack pointer is decremented by one & content of accumulator are copied. The stack pointer is again decremented by one & content of program status word are copied.

No. of bytes : 1

Addressing Mode : Indirect

eg PUSH PSW

Before Execution

[SP] = 7000H

[A] = 55 H

PSW or flag = 44H

After Execution

[6FFF] = 55 H

[6FFE] = 44 H

[SP] = 6FFE H

[A] = 55H

PSW or Flag = 44 H

Scanned with OKEN Scanner

3) POP rp

$[r_L] \leftarrow [[SP]]$

$[r_H] \leftarrow [[SP] + 1]$

$[SP] \leftarrow [SP] + 2$

POP the content of register pair from the stack. The content of memory location pointed by SP, are copied to the lower order register of rp. The SP is incremented by 1 and content of that memory location are copied to higher order register of rp. The SP is again incremented by 1.

No. of Bytes: 1

Addressing Mode: Indirect

eg POP H

Before Execution

[SP] = [6FFE]H

6FFE] = 34 H

6FFF] = 12 H

[HL] = 5678H

After Execution

[SP] = 7000 H

[6FFE] = 34 H

[6FFF] = 12 H

PC = 1234 H

4) POP PSW

[Flag or PSW] ← [[SP]]

[A] ← [[SP] + 1]

[SP] ← [SP] + 2

POP the content of processor status word from the stack. The content of memory location pointed by SP are copied to flag or program status word. The SP is incremented by 1 and content of that location are copied to the accumulator. The SP is incremented by one.

No. of Bytes: 1

Addressing Modes: Indirect

Before Execution: [6FFE]H = 44H

[6FFF] = 55 H

[SP] = 6FFE H

[Flag or PSW] = 12H

[A] = 34H

After Execution: [6FFE] = 44 H

[6FFF] = 55 H

[SP] = 7000 H

[Flag or PSW] = 44 H

[A] = 55 H

Scanned with OKEN Scanner

5) XTHL

[L] ← [[SP]]

[H] ← [[SP] + 1]

Exchange stack top with HL pair. The content of L register are exchanged with stack location pointed by stack pointer. The content of H register are exchanged with the next location. The content of the stack pointer are not affected.

No. of bytes: 1

Addressing Modes: Indirect

eg XTHL

Before Execution [HL] = 1234 H
[SP] = 7000 H
[7000] = 78 H
[7001] = 56 H

After Execution [HL] = 5678 H
[SP] = 7000 H
[7000] = 78 H
[7001] = 56 H

6) SPHL

[SP] ← [HL]

The content of HL pair are moved to the SP register

No. of bytes : 1

Addressing Mode : Register

eg SPHL

Before Execution [SP] = 7000 H
[HL] = 5000 H

After Execution [SP] = 5000 H
[HL] = 5000 H

7) IN port - address

[A] ← [Port]

The content of port is moved to the accumulator. The address of port is 8- bit address.

No. of bytes : 2

Addressing Modes : Direct.

Scanned with OKEN Scanner

eg IN 02

Before Execution [A] = 55 H After Execution [A] = 77 H

[02] = 77 H [02] = 77 H

8) OUT port- address

[Port] ← [A] The content of accumulator is moved to port. The address of port is 8- bit address.

- No. of Bytes : 2

Addressing Mode: Direct

eg OUT 02

Before Execution [A] = 55 H After Execution [A] = 55 H

[02] = 77 H [02] = 55 H

9) HLT

This instruction stops the microprocessor. The register & status flags remain unaffected.

No. of Bytes : 1

Addressing Mode: None

10) NOP No Operation

No Operation is performed.

No. of bytes : 1

Addressing Mode: None

Scanned with OKEN Scanner

6) Interrupt Control Group

The instruction of this group perform interrupt related operations.

1) RST n Restart (n is from 0 to 7)

This instruction transfer the program control to specific memory location known as restart location. This instruction can be used as software interrupt.

No. of Bytes: 1

Addressing Mode: Indirect.

RST	Instruction	Restart Address
	RST 0	0 X 8 = 0000 H
	RST 1	1 X 8 = 0008 H
	RST 2	2 X 8 = 0010 H
	RST 3	3 X 8 = 0018 H
	RST 4	4 X 8 = 0020 H
	RST 5	5 X 8 = 0028 H
	RST 6	6 X 8 = 0030 H
	RST 7	7 X 8 = 0038 H

eg RST 3

Before Execution [PC] = 7000 H
[SP] = 5000 H

After Execution [PC] = 0018 H
[4FFF] = 70 H
[4FFE] = 00 H
[SP] = 4FFE H

Scanned with OKEN Scanner

2) EI (Enable Interrupts)

When this instruction is executed, the interrupt system is enable.

No. of Bytes: 1

Addressing Mode: None

3) DI (Disable Interrupts)

When this instruction is executed, the maskable interrupts are disabled.

No. of Bytes: 1

Addressing Mode: None

4) SIM (Set Interrupt Mask)

→ The execution of SIM enables / disables interrupts according to the bit pattern of accumulator.

→ The instruction is also used for serial data transmission.

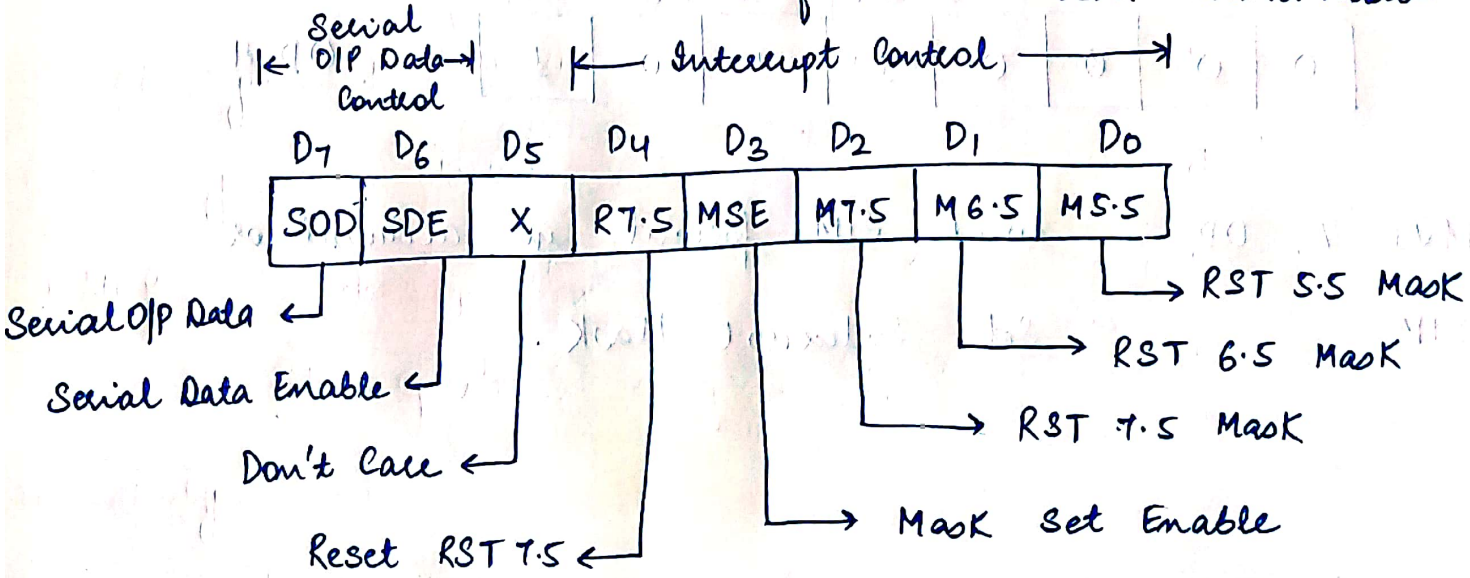


Fig:- SIM format or Accumulator Content for SIM

Scanned with OKEN Scanner

D_0 to D_2 : These bits are used to mask RST 5.5, RST 6.5 and RST 7.5. If a bit is set to 1 the corresponding interrupt is masked or disabled. If it is set to 0, the corresponding interrupt is enabled.

D_3 : It is set to 1 to make D_0 to D_2 effective.

D_4 : If it is set to 1, the flip flop for RST 7.5 is reset.

D_5 : Don't Care

D_6 & D_7 : These bits are for serial data output.

No. of byte : 1

Addressing Mode : None

eg To enable RST 6.5 & disable RST 7.5 and RST 5.5 the following SIM format is required.

SOD	SDE	X	R7.5	MSE	M7.5	M6.5	M5.5
0	0	0	0	1	1	0	1

= OD H

MVI A, OD : Load SIM format in accumulator

SIM : Set Interrupt Mask.

Scanned with OKEN Scanner

5) RIM (Read Interrupt Mask)

When this instruction is executed, the accumulator is loaded with status of pending interrupt, the status of masked interrupt mask and content of SID (Serial Input Data)

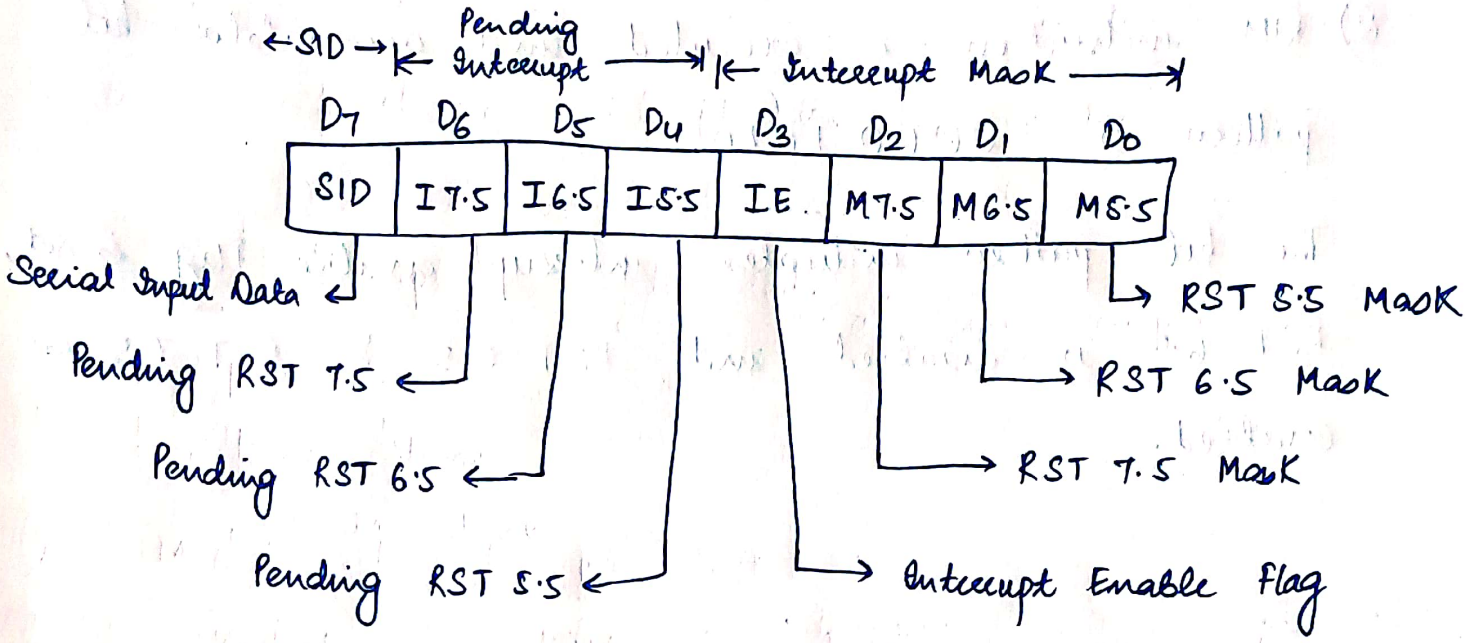


Fig:- RIM format or Accumulator Content after RIM

D_0 to D_2 : These bits give current status of only maskable interrupts.

D_3 : This bit give status of interrupt enable flag.
If it is 1, then interrupt enable flag is set.
If it is 0, then interrupt enable flag is reset.

D_4 to D_6 : These bits gives status of pending interrupt.

D_7 : These bits gives the status of serial input data at SID pin.

No. of Byte: 1

Addressing Mode: None

Scanned with OKEN Scanner

Q) 1) RIM instruction is executed and accumulator bit pattern is 0001 0000.

The bit pattern indicates that RST 5.5 is pending.

2) RIM instruction is executed and accumulator bit pattern is 0000 1010.

The bit pattern indicates interrupt enable flag is set, RST 6.5 is disabled and RST 5.5 & RST 7.5 is enabled.

Scanned with OKEN Scanner

TIMING DIAGRAMS

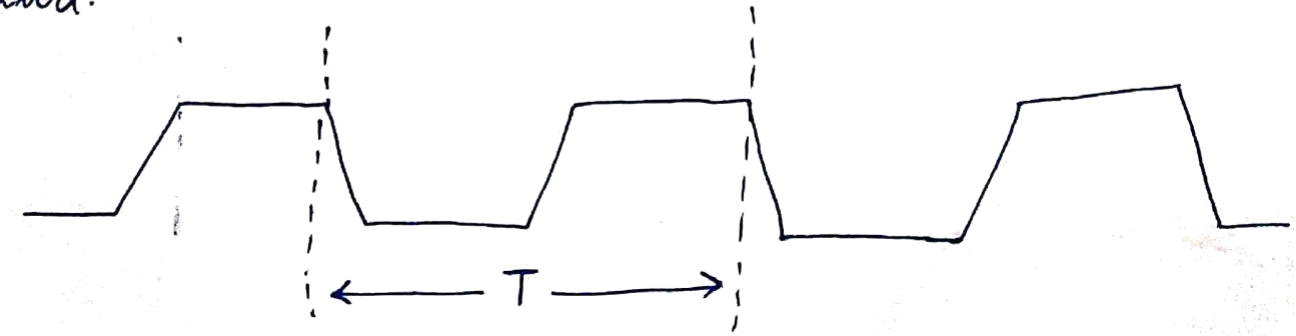
- In programming, the µP fetches and executes instruction one by one till the HLT instruction.
- During fetch and execute operation the timing and control unit of microprocessor generates different timming and control signal i.e CLK, $IO/\overline{M}$, S_1, S_0, ALE, $\overline{RD}$, $\overline{WR}$, $AD_0 - AD_7$, $A_8 - A_{15}$.
- These signals are issued to memory or I/O devices to perform different task. All the operations performed by the microprocessor is in synchronization with clock pulses.
- The graphical representation of steps with respect to time i.e clock is called timing diagram.
- When any instruction is executed by microprocessor, it perform two main operation i.e. Instruction fetch & Instruction Execution

★ T - State:-

- The time between two positive and negative transitions of the clock is defined as T- State.

or

It is one subdivision of the operation performed in one clock period.



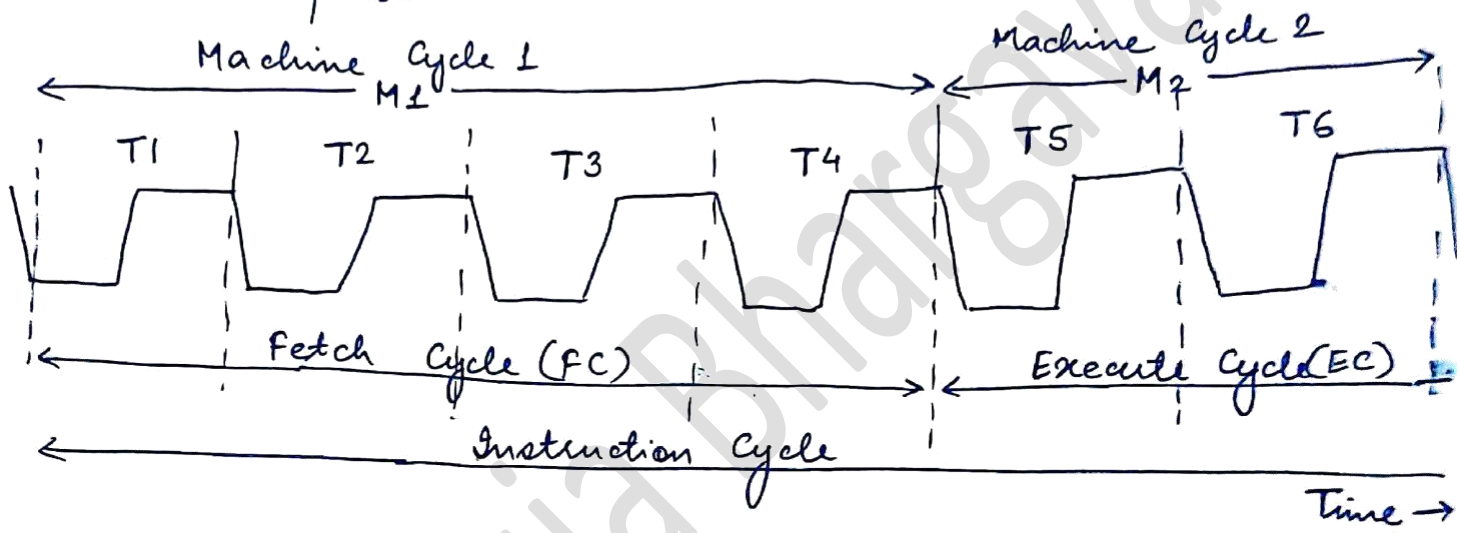
Dr Anuja Bhargava

✱ Instruction Cycle

- It is defined as the time required to complete the execution of an instruction.
- An instruction cycle consist of fetch cycle and execute cycle.

$$IC = FC + EC$$

- The fetch and execute cycle are carried out in synchronization with clock pulses.

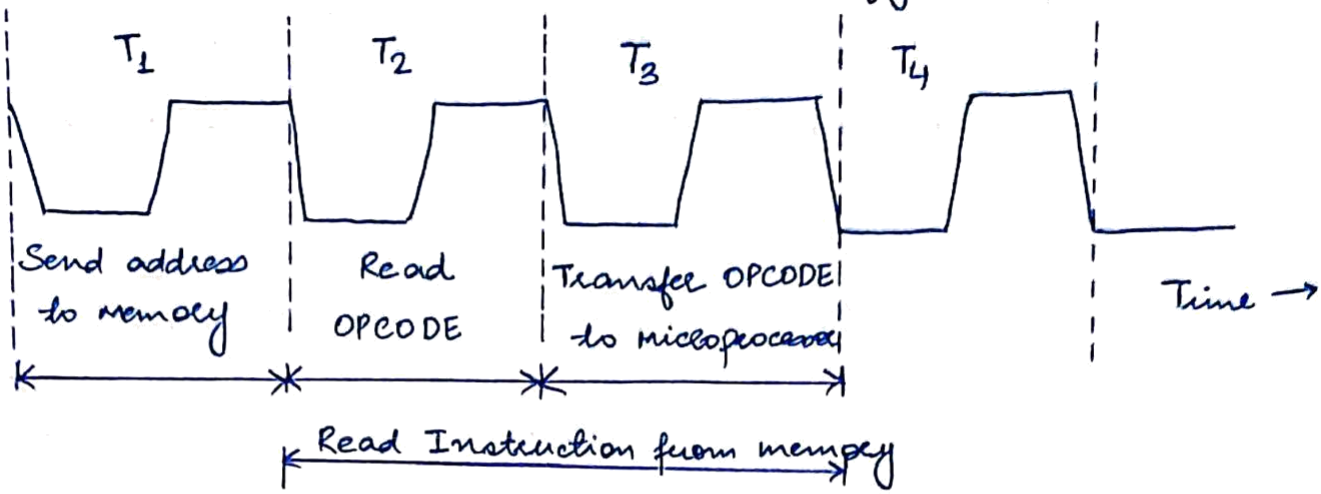


✱ Fetch Cycle

- To execute an instruction, the microprocessor first take the operation code from memory i.e opcode fetch operation.
- The fetch operation is performed in following steps
  - (i) Send instruction of next instruction to memory.
  - (ii) Recieve opcode from memory.
  - (iii) Update program counter.
- Program counter keep tracks of the address of instruction to be fetched next.

Dr Anuja Bhargava

- The above three steps are shown in fig.



★ Execute Cycle

- After receiving the opcode from memory, microprocessor decodes the fetched opcode and then generates timing and control signals to perform a specified task.
- The total time required to do the execute operation is known as execute cycle.
- The time required for execute cycle vary from instruction to instruction.

★ Comparison between fetch and execute cycle.

Fetch Cycle	Execute Cycle.
1. The µP executes this cycle to access program memory.	1. The µP executes this cycle to access data memory.
2. During this cycle, the µP performs only read operation.	2. During this cycle, the µP perform either read or write operation.
3. Number of cycles depends on length of instruction.	3. Number of cycle depends on type of instruction, addressing mode.
4. Program counter is incremented by 1.	4. Program counter is not incremented.
5. Content of PC are placed on address bus.	5. Content of SP are placed on address bus.

Dr Anuja Bhargava

✱ Machine Cycle

- The time required by the microprocessor to complete the operation i.e memory read / write or I/O read / write is called as machine cycle.
- The 8085 instruction consist of 1 to 5 machine cycles. The first machine cycle of every instruction is opcode fetch cycle followed by memory read or memory write or I/O read or I/O write.

OPCODE FETCH CYCLE :-

- The first byte of any instruction is opcodes or operation code.
- The microprocessor uses this cycle to fetch opcode from memory known as opcode fetch cycle.
- In this, the microprocessor place content of program counter i.e address of memory on address bus. The program counter is incremented by one.
- During this cycle, the microprocessor fetches opcode of current instruction into instruction register and then decodes this opcode.

Note:- The most of the 8085 microprocessor requires 4T states opcode fetch cycle except CALL, DCX, INX, PCHL, PUSH, SPHL, condition RET and RSTn instructions.

Dr Anuja Bhargava

Timing & Control Signal :-

ALE :- Address Latch Enable

It indicates that bits on $AD_7 - AD_0$ are address bits.

$\overline{RD}$:- Read (Active low)

It indicates that selected I/p or memory device is to be read and data are available on data bus.

$\overline{WR}$:- Write (Active low)

It indicates that the data on data bus are to be written into selected memory or I/O location.

$IO/\overline{M}$

When it is high, it indicates I/O operation.

" " " low ; " " memory "

S_1, S_0

To identify various operations.

CLK

This is used as system clock.

Dr Anuja Bhargava

Machine Cycle	Status $IO/\overline{M}$	S_1	S_0	Control Signal
Opcode fetch	0	1	1	$\overline{RD} = 0$
Memory Read	0	1	0	$\overline{RD} = 0$
Memory Write	0	0	1	$\overline{WR} = 0$
I/O Read	1	1	0	$\overline{RD} = 0$
I/O Write	1	0	1	$\overline{WR} = 0$

Microprocessor Communication

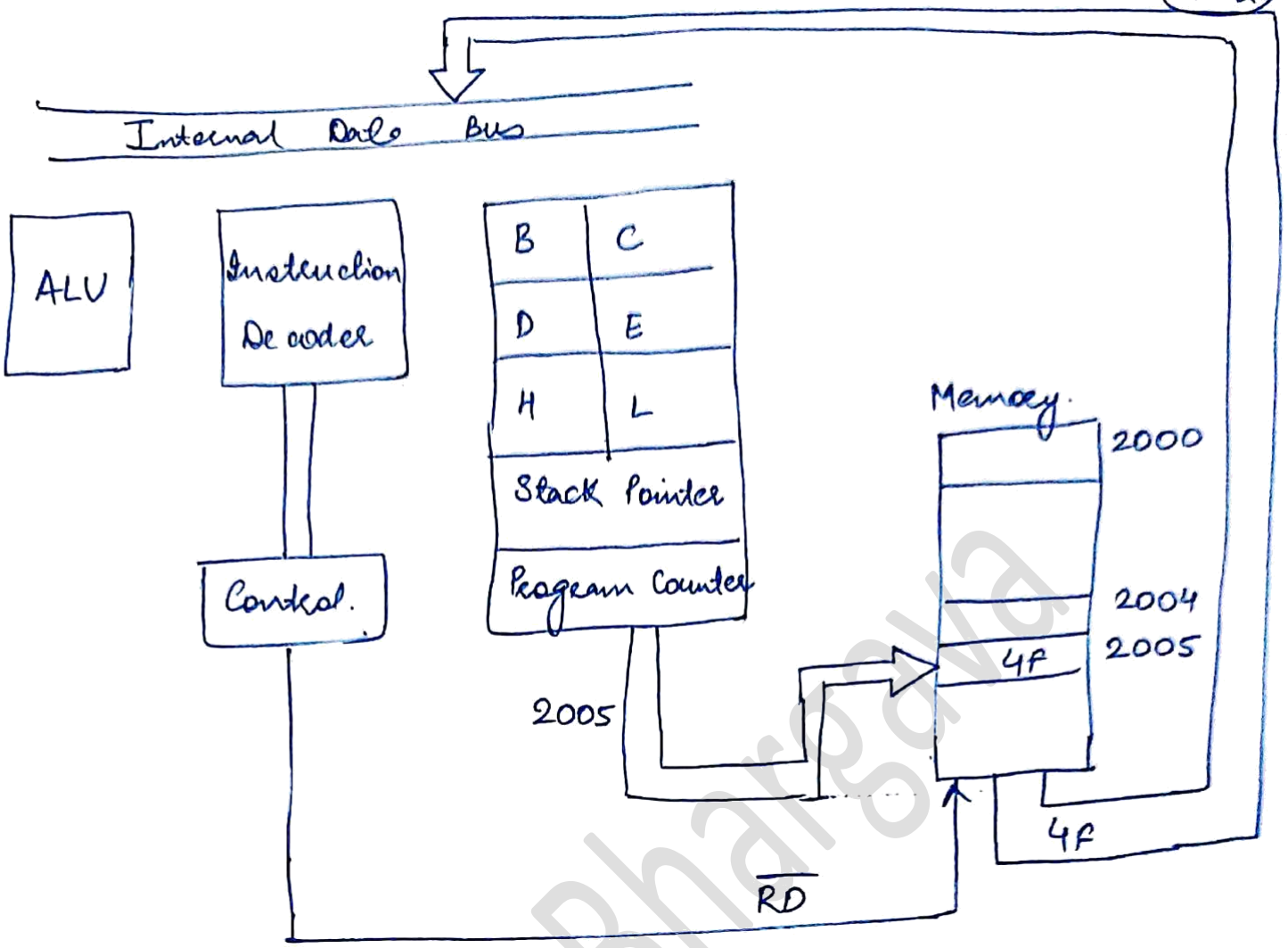
To understand the function of various signal of 8085, we examine the communication between microprocessor and I/O devices.

Step (1) Reading from memory or fetching the instruction.

Step (2) This can be understood by analogy of courier.

1) Courier gets the address from office, picks the van, find the street and look for house number.
2) Courier rings the bell.
3) Somebody opens the door, and gives package by courier.
4) Return back.

Dr Anuja Bhargava



Step 1) µP places the 16 bit memory address from the program counter on the address bus.

(Courier finds the street and address)

At T_1, the higher order memory address 20H is placed on A_8-A_{15}

lower " " " 05H " " " A_0-A_7

ALE goes high.

IO/$\overline{M}$ goes low i.e it is memory related operation

S_1 S_0 → 0 1

Step 2) Control unit send the control signal $\overline{RD}$ to enable the memory. (Ringing the bell)

Dr Anuja Bhargava

At T_2, $\overline{RD}$ signal is send out, active for two clock periods.

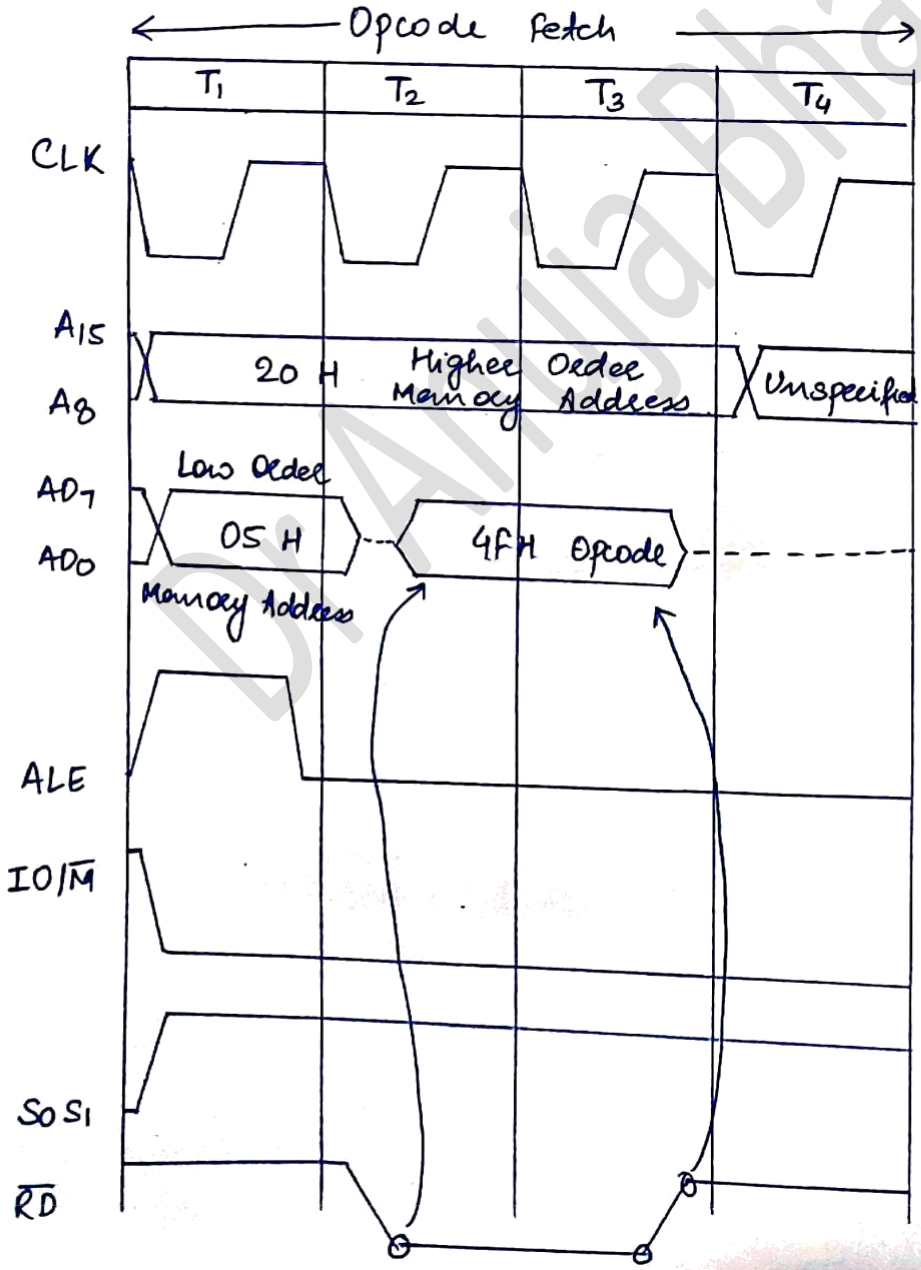
Step 3) The byte from memory location is placed on data bus.

When memory is enable, 4FH is placed on $AD_7 - AD_0$.

The $\overline{RD}$ signal causes 4FH to be placed on $AD_7 - AD_0$.

Step 4) The byte is placed in instruction decoder of the Microprocessor and the task is carried out according to the instruction.

The 4FH is decoded, at T_4.



Dr Anuja Bhargava

Memory Read (Operand Fetch Cycle)

- The microprocessor execute this cycle to read contents from data memory. The length of cycle is 3T state.
- Operation

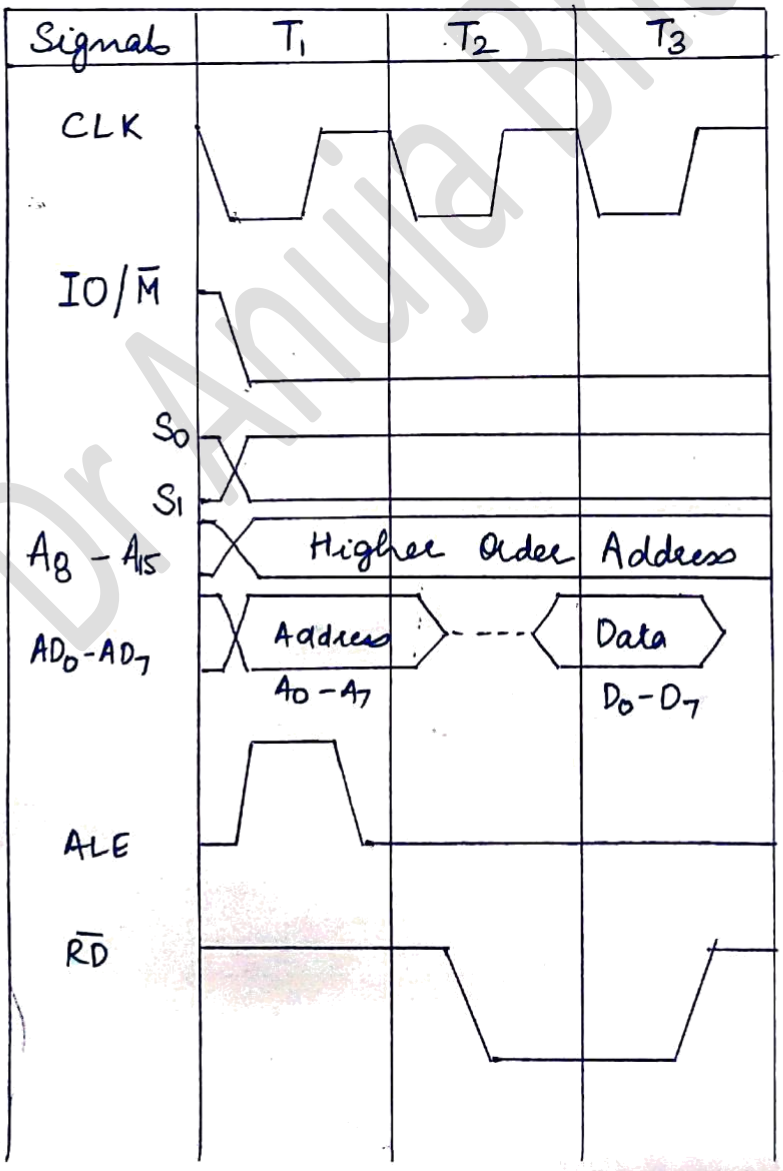
1. T_1 State:

In this state, 8085 sends $IO/\overline{M} = 0$, $S_0 = 0$ & $S_1 = 1$.

In memory read, address is from WZ or SP.

In operand fetch cycle, address is given by PC and incremented by 1

During this state, ALE is enable to enable address at A_0-A_7.



Dr Anuja Bhargava

2. T_2 - State

In this state, $AD_0 - A_7$ lines will be used as $D_0 - D_7$ ie data lines.

During this state $\overline{RD} = 0$, read is enabled and content of selected memory location will be placed on $D_0 - D_7$.

3. T_3 - State

During this state, data from memory is transfered to microprocessor & $\overline{RD}$ is made high. and PC is incremented by 1.

Memory Write Cycle

- The microprocessor execute this cycle to write data in memory.
- During this cycle, the content of register are placed on the address bus. Program counter is not incremented by one.
- The length of cycle is 3T state.
- Operation

1) T_1 - State :

In this state, 8085 sends $IO/\overline{M} = 0$, $S_0 = 1$, $S_1 = 0$.

The 8085 place the content of register on address bus.

ALE is enable to activate address $A_0 - A_7$.

2) T_2 - State

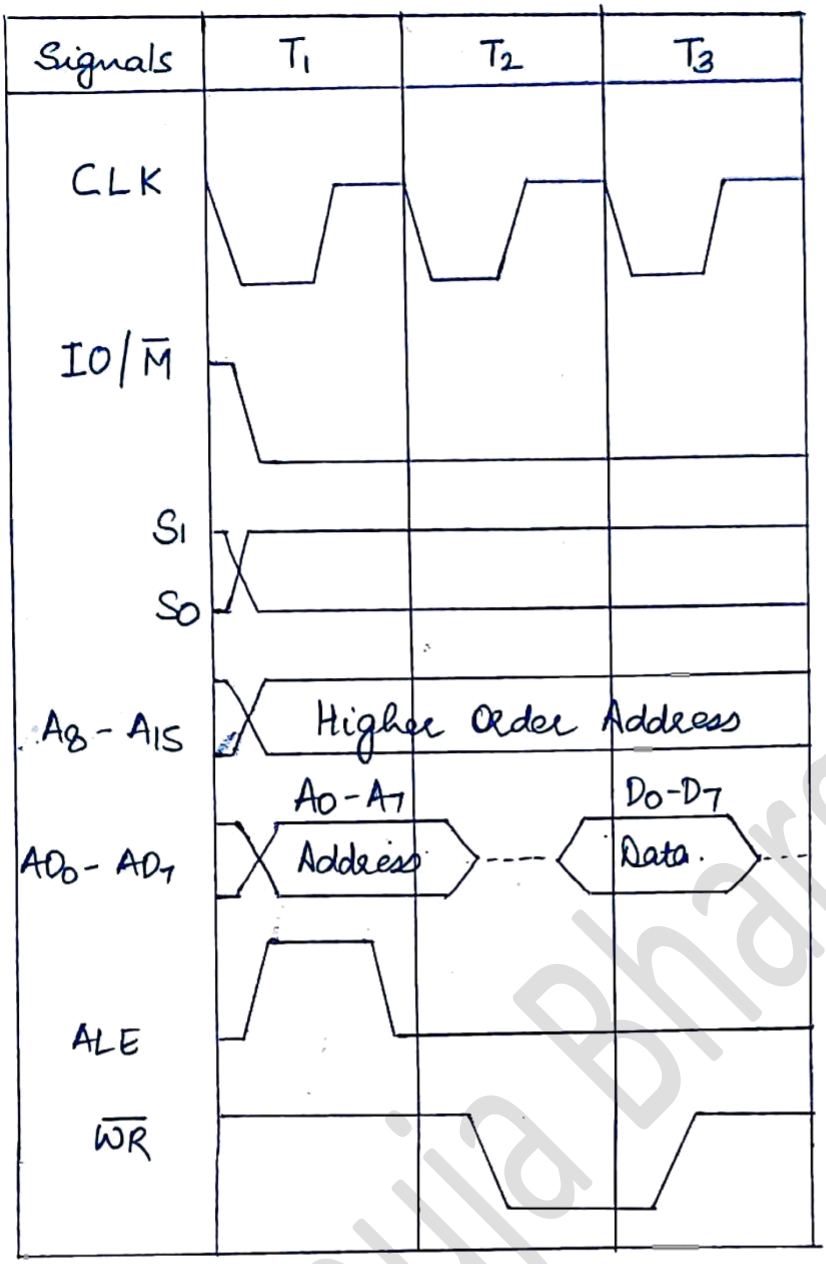
In this state, the $AD_7 - AD_0$ will be used as $D_0 - D_7$ i-e data lines

During this state $\overline{WR} = 0$, the write circuit of memory device is enabled to write and the content of data bus will be written on memory location.

3) T_3 - State

During this state, the content of data bus is stored in memory location and $\overline{WR}$ is made high.

Dr Anuja Bhargava



Input / Output Read Cycle

- The microprocessor executes this cycle to read content of I/O port
- The length of the cycle is 3T states.

T_1 State :-

In this state, the 8085 sends signals IO / $\overline{M}$ = 1, S_0 = 0 and S_1 = 1. The content is placed on address bus.

During this state, the ALE pulse is activated to enable address latch $A_0 - A_7$.

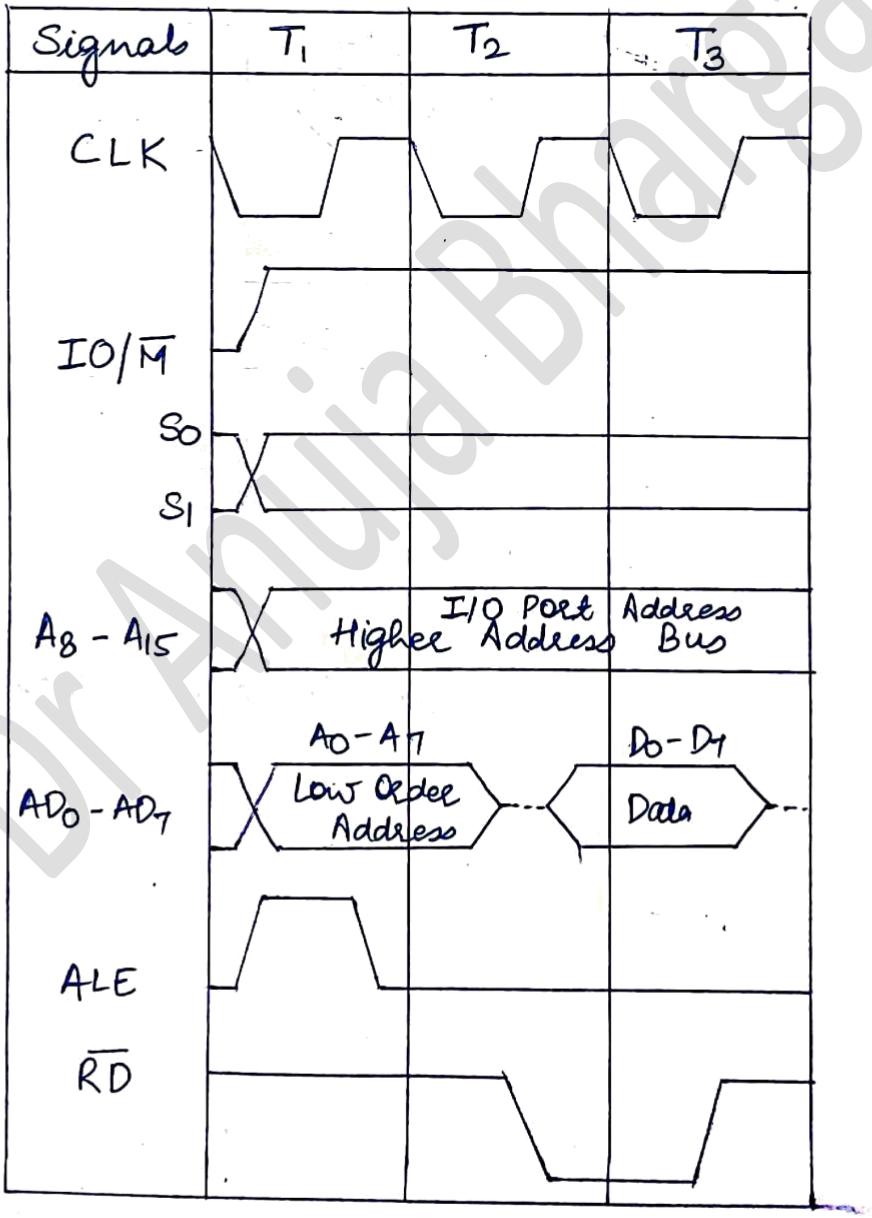
Dr Anuja Bhargava

T_2 State :-

In this state, the $AD_0 - AD_7$ lines will be used as $D_0 - D_7$ i-e data bus. During this state $\overline{RD} = 0$, the read circuit of I/O device is enable to read and the content of selected I/O port is placed on data bus.

T_3 State :-

During this state, the content of I/O port is transfered to Microprocessor. i-e A register. and $\overline{RD}$ is made high.



Dr Anuja Bhargava

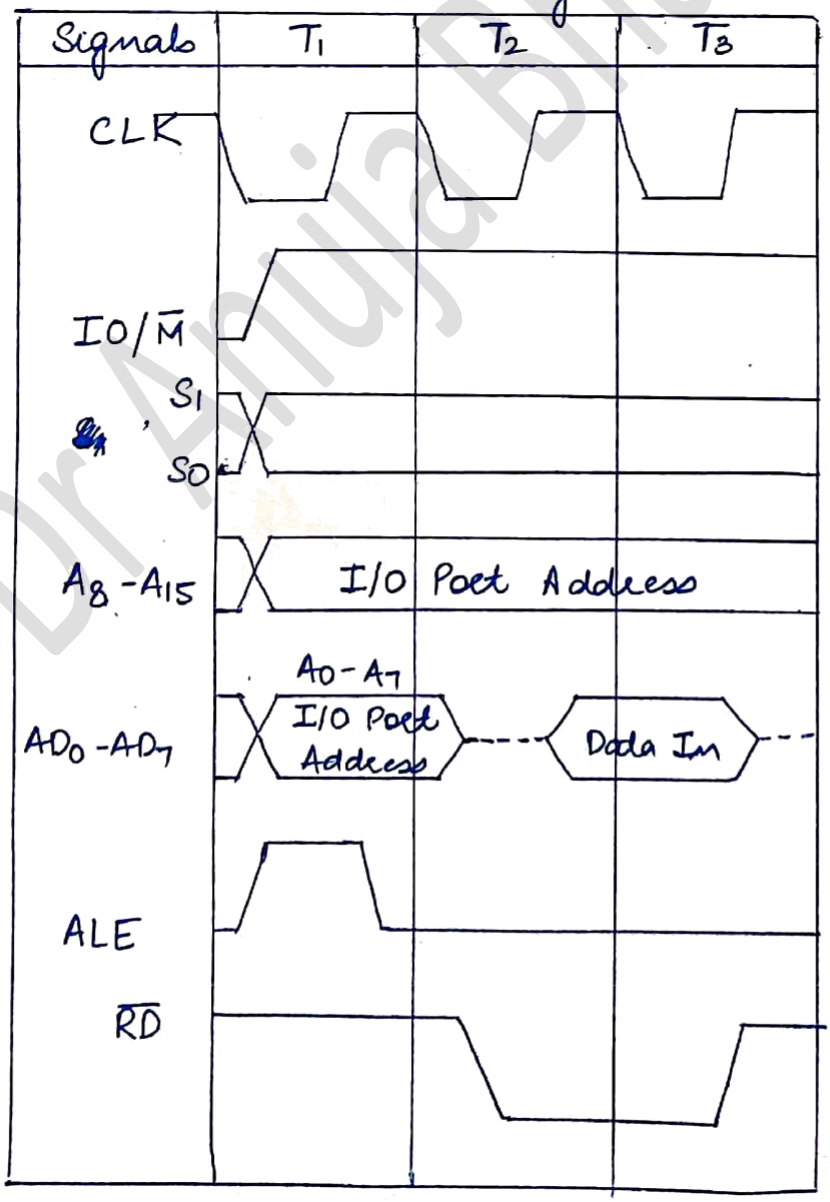
Input / Output Write Cycle

- The microprocessor executes this cycle to write data into an I/O port.
- The length of the cycle is 3T States.

T_1 State:- In this state, the 8085 sends signals $IO/\overline{M}$, $S_0 = 1$, $S_1 = 0$. In this cycle, the content of register is placed on address bus. The same 8 bit I/O address is placed on $A_0 - A_7$ & $A_8 - A_{15}$. During this state, the ALE is activated.

T_2 State:- In this state, $AD_0 - AD_7$ lines will be used as $D_0 - D_7$ i.e data lines. During this state $\overline{WR} = 0$, write is enabled and content is placed on data bus.

T_3 State:- During this state the content of accumulator is written to I/O port and $\overline{WR}$ is made high.



Dr Anuja Bhargava

8.10 TIMING DIAGRAMS FOR 8085 INSTRUCTION

The instructions of 8085 are one, two or three byte instructions. The timing diagram of each instruction is combination of opcode fetch cycle, memory read cycle, memory write cycle, I/O read cycle, I/O write cycle and INTA cycle. The first machine cycle in each instruction is opcode fetch cycle of length 4T or 6T states. The number of machine cycles required for each instruction depends up on type of instruction, and availably of operand i.e. in memory or registers.

After fetching the instruction it requires additional machine cycles if the operand is available in memory. The minimum number of machine cycles or T states required for one, two or three byte instruction is given in table 8.5.

Table 8.5.

Type of Instruction	Minimum		Additional		Total	
	Machine Cycles	T states	Machine Cycles	T states	Machine Cycles	T states
One Byte	1 (OF)	4/6	Operand Availibity	Operand Availibity		
For example						
MOV A, B	1 (OF)	4	-	-	1	4
MOV A, M	1 (OF)	4	1 (MR)	3	2	7
Two Byte	1 + 1 (OF) (OPF/MR)	4/6 + 3	Operand Availibity	Operand Availibity		
For example						
MVI A, 02	1 + 1 (OF) (OPF/MR)	4 + 3	-	-	2	7
MVI M, 02	1 + 1 (OF) (OPF/MR)	4 + 3	1 (MW)	3	2	10
Three Byte	1 + 1 + 1 (OF) (OPF/MR) (OPF/MR)	4/6 + 3 + 3	Operand Availibity	Operand Availibity		
For example						
LX14, 7000	1 + 1 + 1 (OF) (OPF/MR) (OPF/MR)	4 + 3 + 3	-	-	3	10
STA 7000	1 + 1 + 1 (OF) (OPF/MR) (OPF/MR)	4 + 3 + 3	1 (MW)	3	4	13

8.10.1 MOV r_1, r_2

This instruction copy the content of r2 register to r1 register.

$$(r_1) \leftarrow (r_2)$$

Instruction	Minimum		Additional		Total	
	Machine cycles	T states	Machine cycles	T states	Machine cycles	T states
One byte	1 (OF)	4	-	-	1	4

Only opcode fetch machine cycle is required to fetch and execute this instruction. Assume that the instruction is stored at memory location 7000H.

7000 Opcode of MOV r_1, r_2 Opcode fetch

The timing diagram is shown in fig. 8.12.

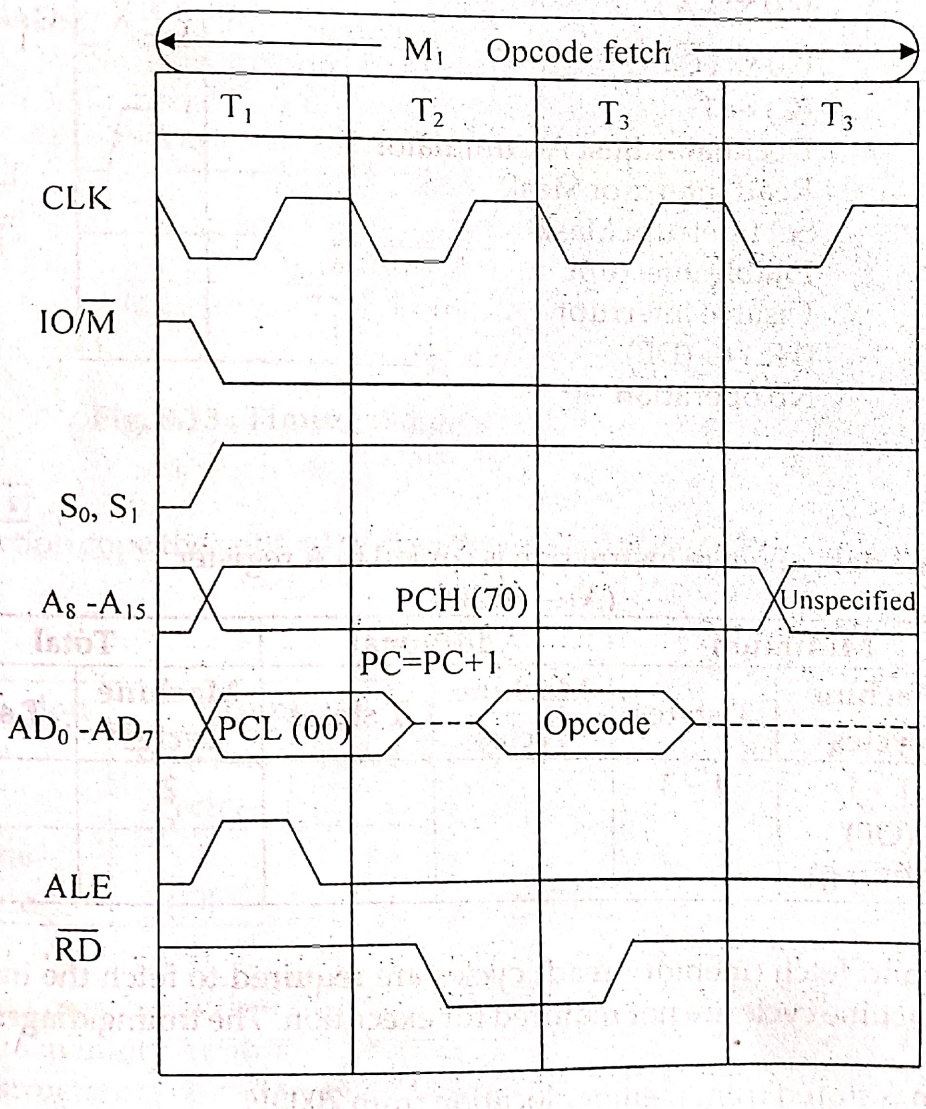


Fig. 8.12 : Timing diagram MOV r_1, r_2 instruction

The timing diagram for following instruction is same as fig. 8.12. These instruction are one byte instruction and doesnot require T5 and T6 states. No additional machines cycles are not required for execution.

8.10.2 MVI A, data :

The 8-bit immediate data given in instruction is copied to A register

(A)← (Data)

Instruction	Minimum		Additional		Total	
	Machine cycles	T states	Machine cycles	T states	Machine cycles	T states
Two Byte	1 + 1 (OF) (OPF/MR	4 + 3	-	-	2	7

Opcode fetch and operand fetch (memory read) cycles are required to fetch the instruction from memory. Additional machine cycle are not required for execution. The timing diagram is shown in fig. 8.13.

Assume that instruction is stored from memory location from 7000H.

For example : MVI A, 25

7000	3E (Opcode)	Opcode Fetch
7001	25 (Data)	Operand Fetch or Memory Read

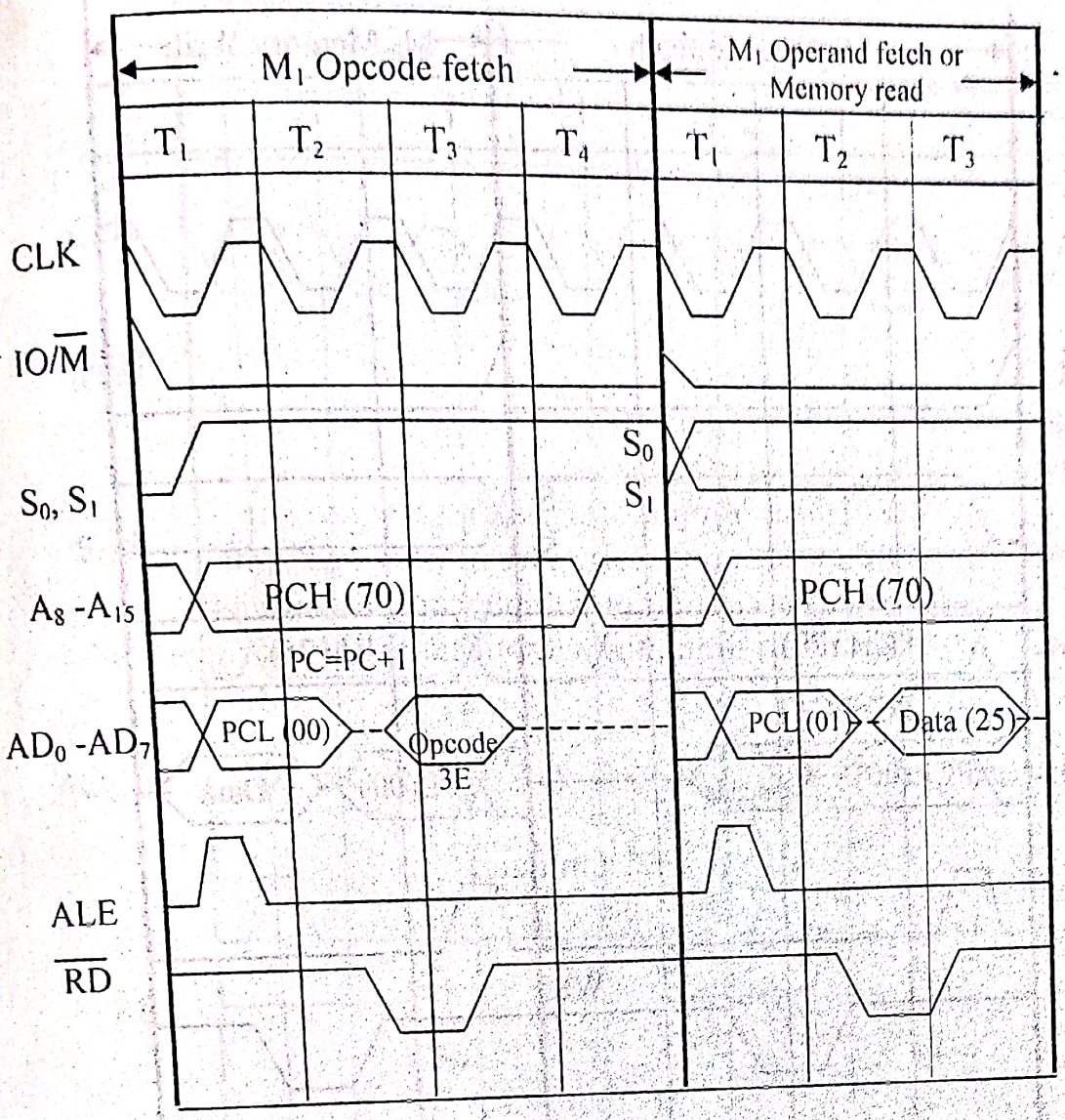


Fig. 8.13 : Timing diagram for MVI A, data instruction

8.10.3 MOV M, r

This instruction cope the content of register r to memory location addressed by HL pair

$$[[HL]] \leftarrow (r)$$

Instruction	Minimum		Additional		Total	
	Machine cycles	T states	Machine cycles	T states	Machine cycles	T states
One Byte	1 (OF)	4	1 (MW)	3	2	7

Opcode fetch and memory write cycles are required. Additional machine cycle is required to copy the content of r to memory location. The timing diagram is shown in fig. 8.14.
Assume that instruction is stored from memory location from 7000H.

For Example : MOV M, A

Assume HL = 7500

7000	Opcode (7E)	Opcode Fetch
7500	Data	Memory Write

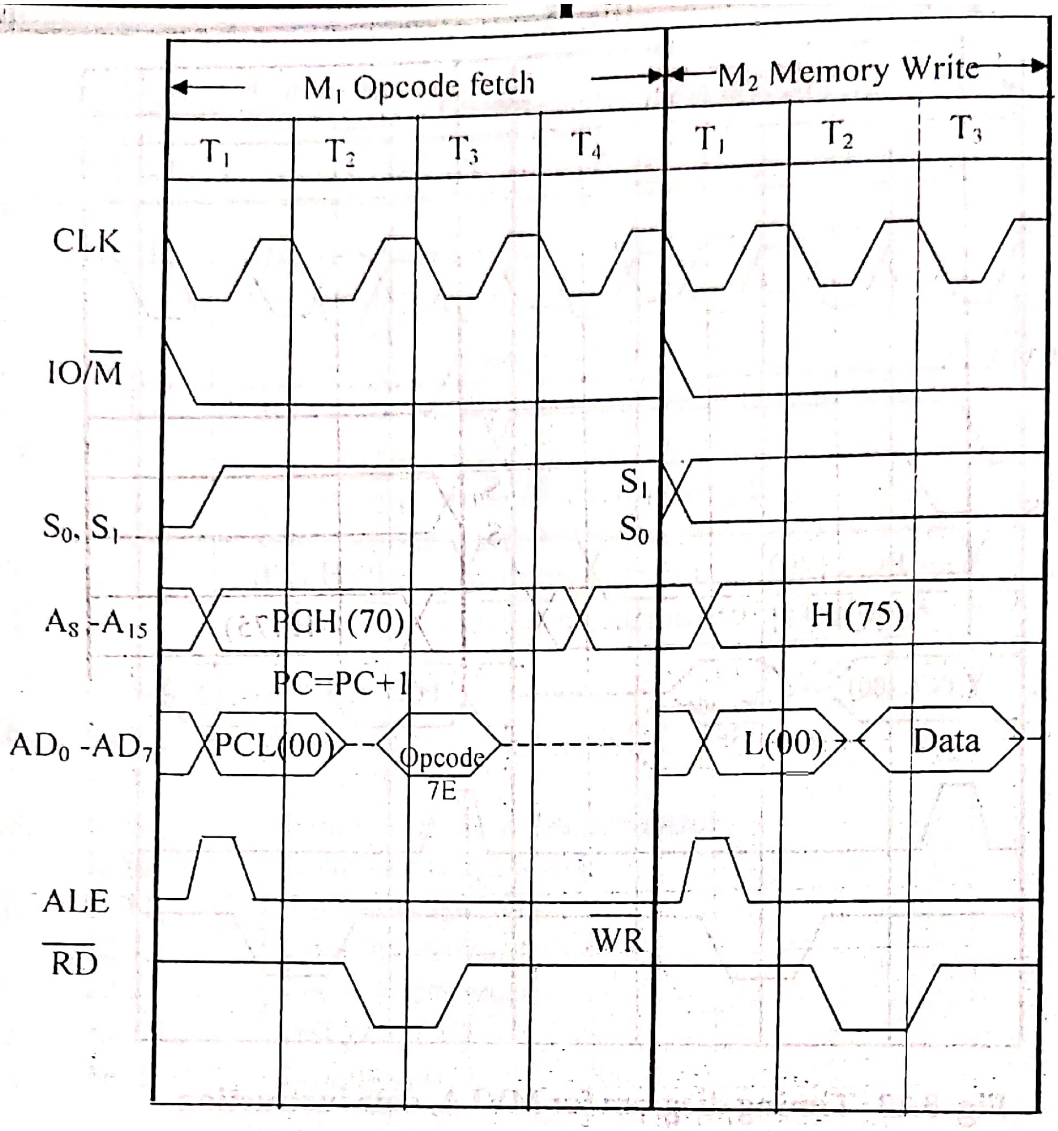


Fig. 8.14 : Timing diagram for MOV M, r instruction

8.10.4 MOV r, M

This instruction copy the content of memory location addressed by the HL pair to register r.

$$(r) \leftarrow [(HL)]$$

Instruction	Minimum		Additional		Total	
	Machine cycles	T states	Machine cycles	T states	Machine cycles	T states
One Byte	1 (OF)	4	1 (MR)	3	2	7

Opcode fetch and memory read cycles are required. Additional machine cycle is required to copy the content of memory location to register. The timing diagram is shown in fig. 8.15. Assume that instruction is stored from memory location from 7000H.

For Example : MOV A, M

Assume HL = 7500

7000	Opcode (77)	Opcode Fetch
7500	Data	Memory Read

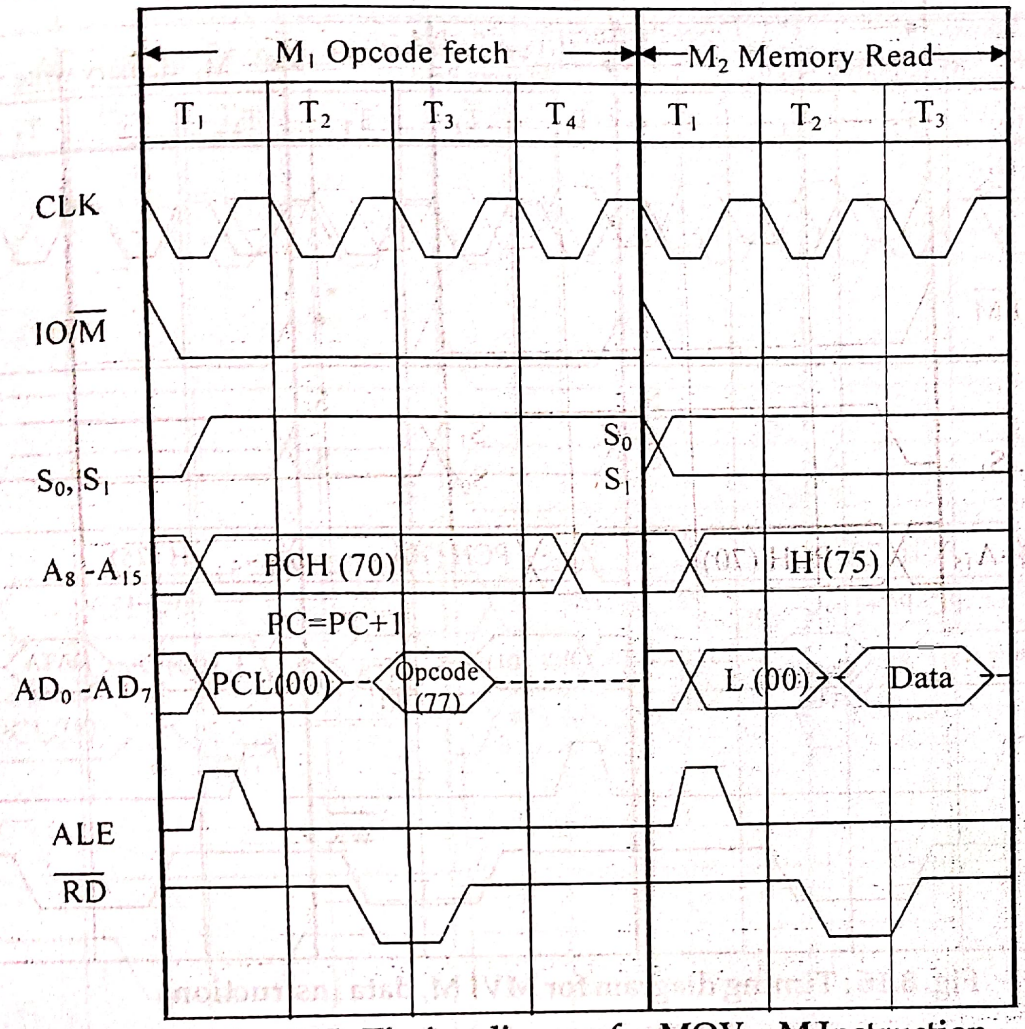


Fig. 8.15 : Timing diagram for MOV r, M Instruction.

8.10.5 MVI M, data

This instruction copy the data given in instruction to memory location addressed by the HL pair.

[(HL)] ← [(Data)]

Instruction	Minimum		Additional		Total	
	Machine cycles	T states	Machine cycles	T states	Machine cycles	T states
Two byte	1 + 1 (OF) (OPF/MR)	4 + 3	1 (MW)	3	3	10

The two machine cycle i.e. opcode fetch and operand fetch (memory read) cycles are required to fetch instruction from memory. Additional machine cycle is required to copy the data to memory location. The timing diagram is shown in fig. 8.16.

Assume that instruction is stored from memory location from 7000H.

For example : MVI M, 55

Assume HL = 7500

7000	Opcode (36)	Opcode Fetch
7001	55	Operand Fetch/ Memory Read
7500	Data	Memory Write

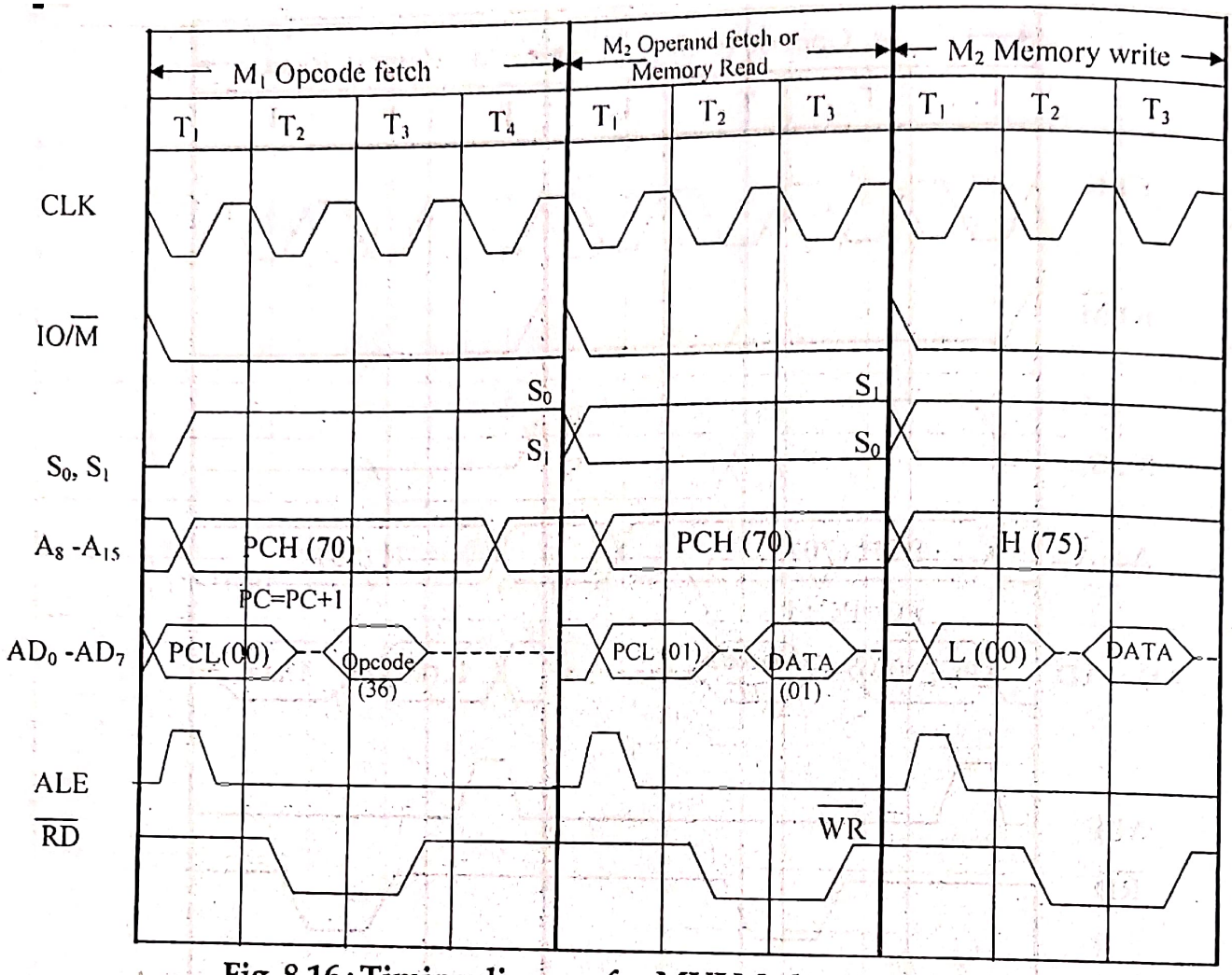


Fig. 8.16 : Timing diagram for MVI M, data instruction

8.10.6 LXI rp, data 16

The 16 bit data specified in the instruction is copied to rp.

(rp) ← Data 16

Instruction	Minimum		Additional		Total	
	Machine cycles	T states	Machine cycles	T states	Machine cycles	T states
Three Byte	1 + 1 + 1 (OF) (OPF/MR) (OPF/MR)	4 + 3 + 3	-	-	3	10

The three machine cycles are required to fetch instruction from memory. Additional machine cycle is not required for execution. The timing diagram is shown in fig. 8.17. Assume that instruction is stored from memory location 7000H.

For example : LXI H, 7500

7000	opcode (21)	Opcode Fetch
7001	low order data (00)	Operand Fetch/ Memory Read
7002	high order data (75)	Operand Fetch/ Memory Read

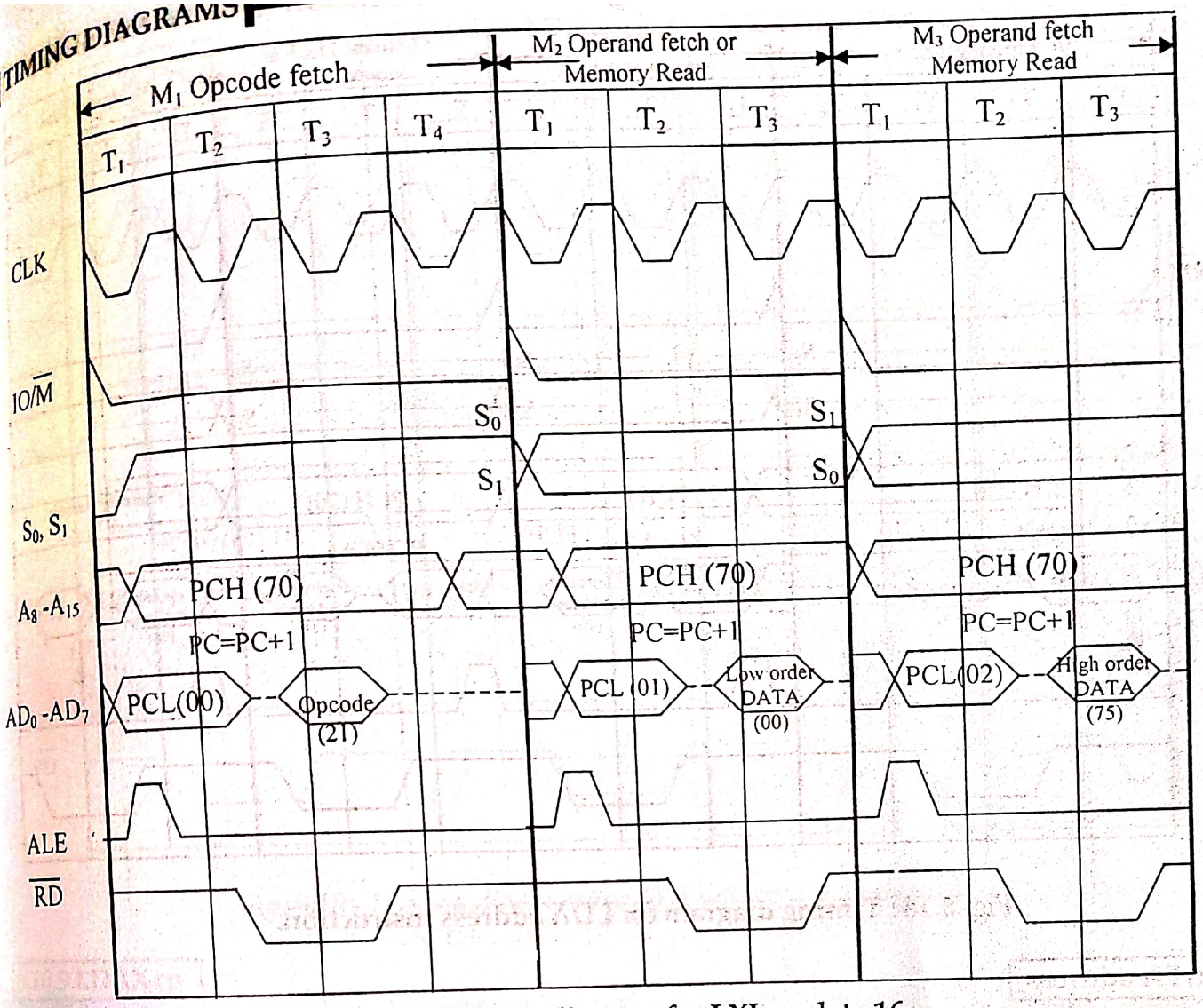


Fig. 8.17 : Timing diagram for LXI rp, data 16

8.10.7 LDA address

This instruction loads the content of memory location whose address is specified in instruction to A register.

(A) ← (address)

Instruction	Minimum		Additional		Total	
	Machine cycles	T states	Machine cycles	T states	Machine cycles	T states
Three Byte	1+1+1 (OF) (OPF/MR) (OPF/MR)	4+3+3	1 (MR)	3	4	13

The three machine cycles are required to fetch instruction from memory. Additional machine cycle is required to load content of memory to A register. The timing diagram is shown in fig. 8.18. Assume that instruction is stored from memory location 7000H.

For example : LDA 7500

7000	Opcode	Opcode Fetch
7001	Lower order address	Operand Fetch/ Memory Read
7002	Higher order address	Operand Fetch/ Memory Read
7500	Data	Memory Read.

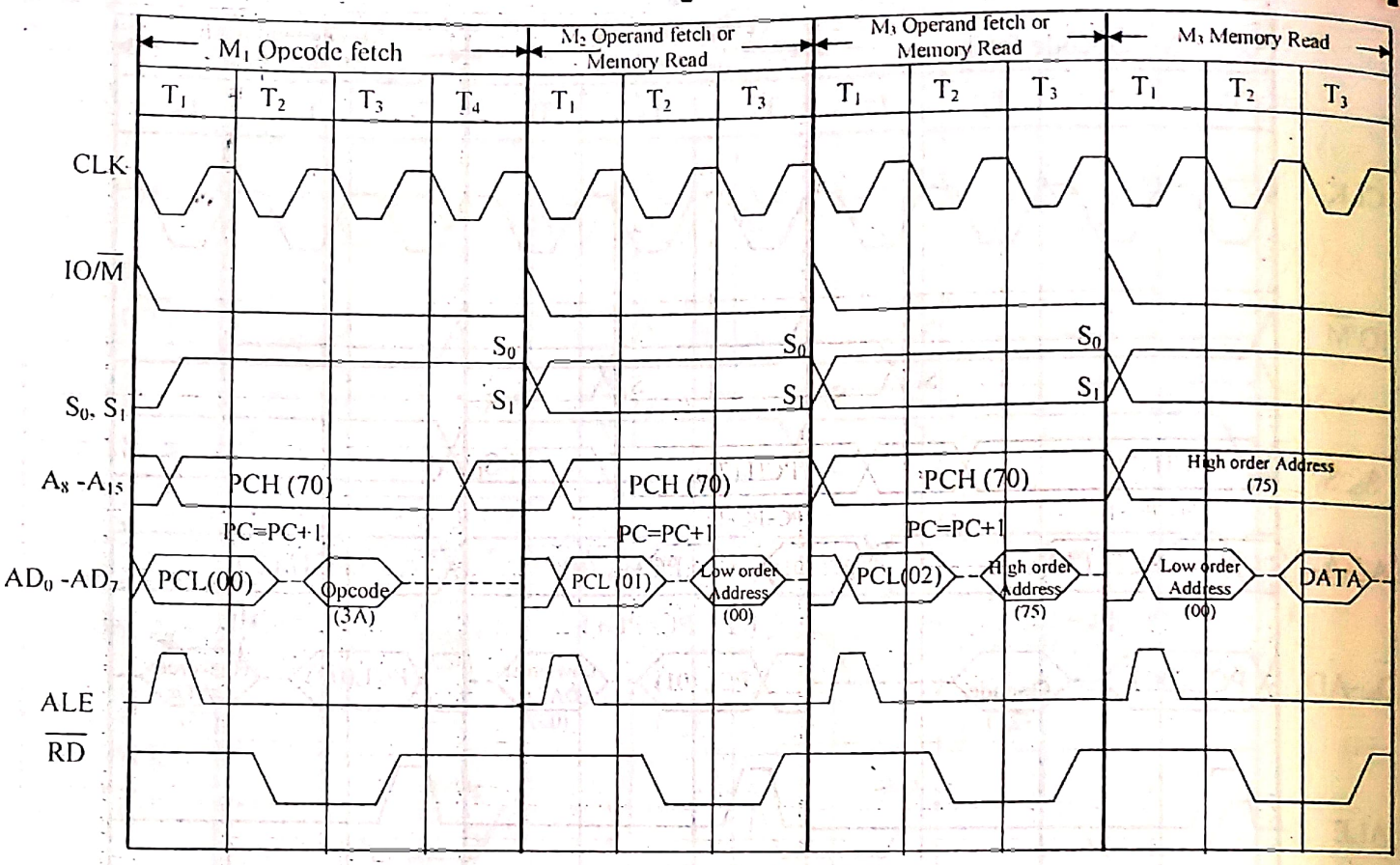


Fig. 8.18 : Timing diagram for LDA address instruction.

8.10.8 STA address

This instruction stores the content of A register to memory location whose address is specified in instruction.

(Address) ← (A)

Instruction	Minimum		Additional		Total	
	Machine cycles	T states	Machine cycles	T states	Machine cycles	T states
Three Byte	1 + 1 + 1 (OF) (OPF/MR) (OPF/MR)	4 + 3 + 3	1 (MW)	3	4	13

The three machine cycles are required to fetch instruction from memory. Additional machine cycle is required to store the content of A register to memory. The timing diagram is shown in fig. 8.19. Assume that instruction is stored from memory location 7000H.

For example : STA 8000

7000	Opcode (32)	Opcode Fetch
7001	Lower order address	Operand Fetch/ Memory Read
7002	Higher order address	Operand Fetch/ Memory Read
8000	Data	Memory Write.

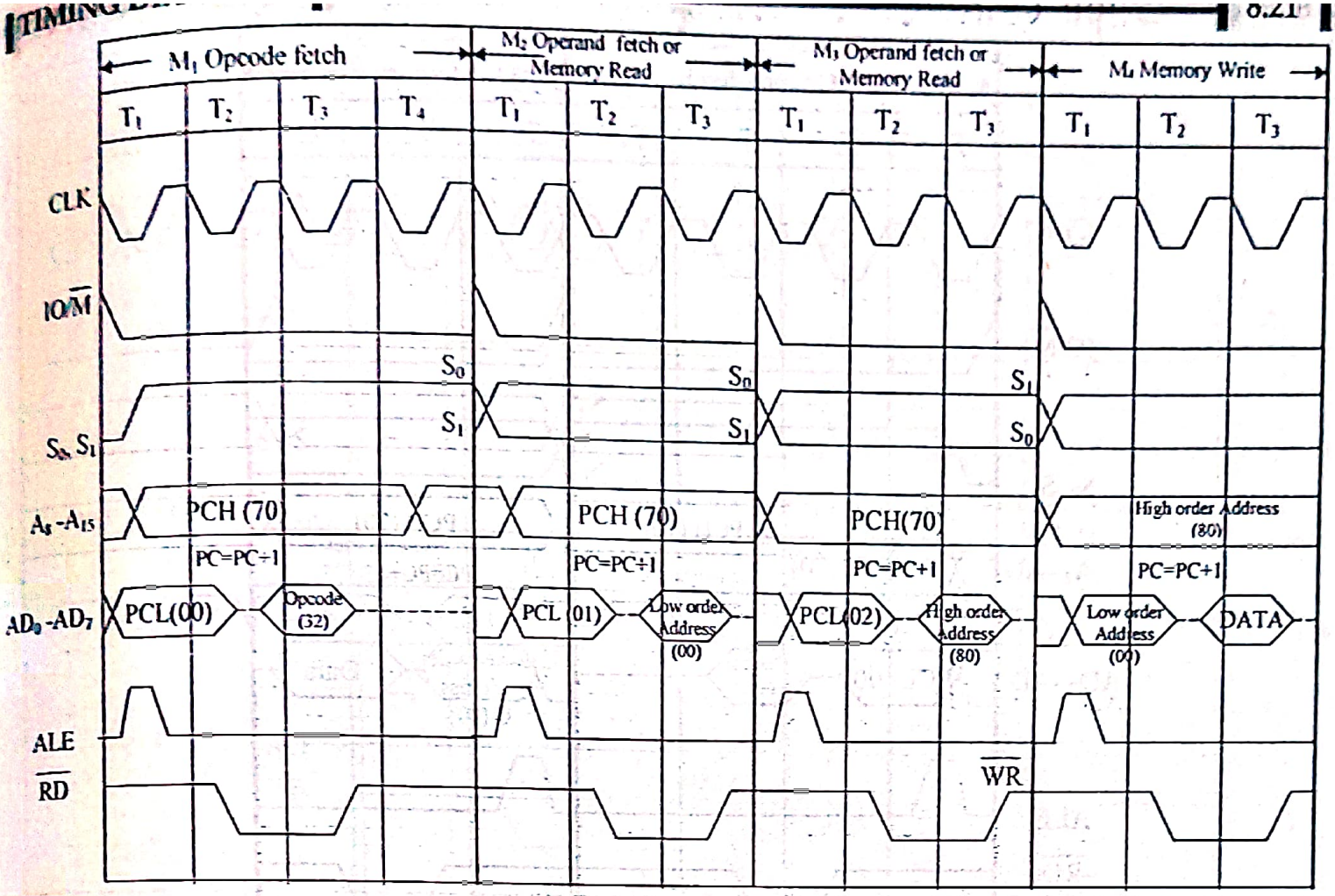


Fig. 8.19 : Timing diagram for STA address instruction.

8.10.9 LDAX rp

This instruction transfers the content of memory location addressed by register pair rp to accumulator.

$$(A) \leftarrow [(rp)]$$

Instruction	Minimum		Additional		Total	
	Machine cycles	T states	Machine cycles	T states	Machine cycles	T states
One Byte	1 (OF)	4	1 (MR)	3	2	7

Only one machine cycle is required to fetch instruction from memory. Additional machine cycle is required to load data from memory to accumulator. The timing diagram is shown in fig. 8.20. Assume that instruction is stored from memory location 7000H.

For Example : LDAX B

Assume BC = 7500

7000	opcode (0A)	Opcode Fetch
7500	Data	Memory Read

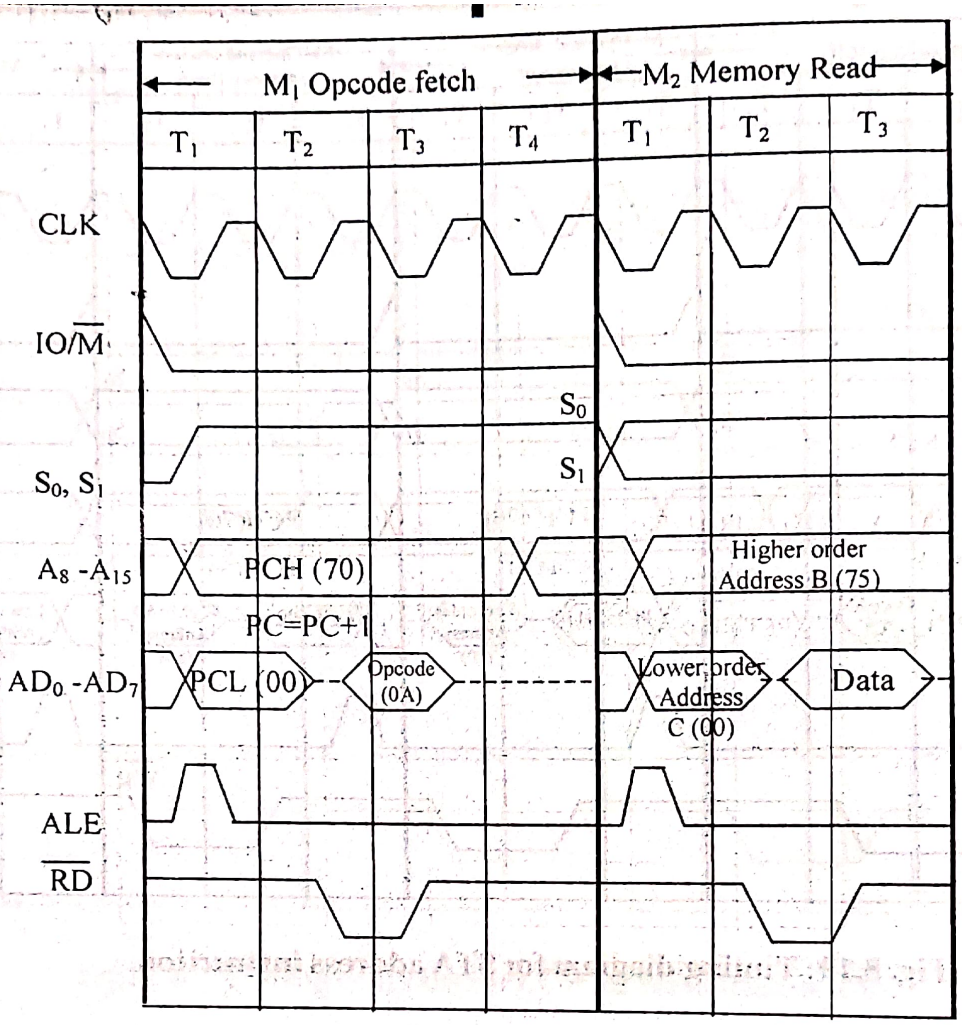


Fig. 8.20 : Timing diagram for LADX rp Instruction.

8.10.10 STAX rp

This instruction store the contents of accumulator to memory location addressed by register pair rp.

$$[(rp)] \leftarrow (A)$$

Instruction	Minimum		Additional		Total	
	Machine cycles	T states	Machine cycles	T states	Machine cycles	T states
One Byte	1 (OF)	4	1 (MW)	3	2	7

One machine cycle is required to fetch instruction from memory. Additional machine cycle is required to store the content of accumulator to memory. The timing diagram is shown in fig. 8.21. Assume that instruction is stored from memory location 7000H.

For Example : STAX D

Assume DE = 8000

7000	opcode (12)	Opcode Fetch
8000	Data	Memory Write

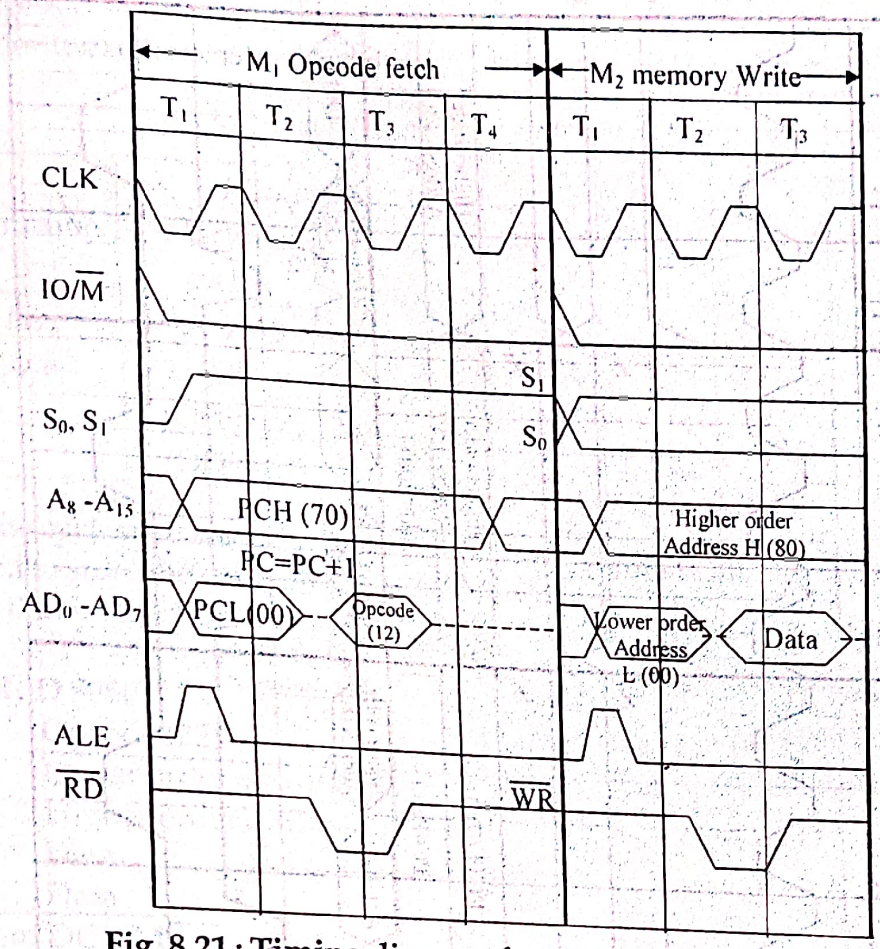


Fig. 8.21 : Timing diagram for STAX rp Instruction.

8.10.11 LHLD address

This instruction load the content of memory location to HL pair.

(L) ← (Address)

(H) ← (Address + 1)

Instruction	Minimum		Additional		Total	
	Machine cycles	T states	Machine cycles	T states	Machine cycles	T states
Three byte	1 + 1 + 1 (OF) (OPF/MR) (OPF/MR)	4 + 3 + 3	1 + 1 (MR) (MR)	3 + 3	5	16

Three machine cycles are required to fetch instruction from memory. Two additional machine cycles are required to load the content of memory location to HL pair. The timing diagram is shown in fig. 8.22. Assume that instruction is stored from memory location 7000H.

For example : LHLD 8050

7000	Opcode (2A)	Opcode Fetch
7001	Lower order address (50)	Operand Fetch/ Memory Read
7002	Higher order address (80)	Operand Fetch/ Memory Read
8050	Data	Memory Read.
8051	Data	Memory Read.

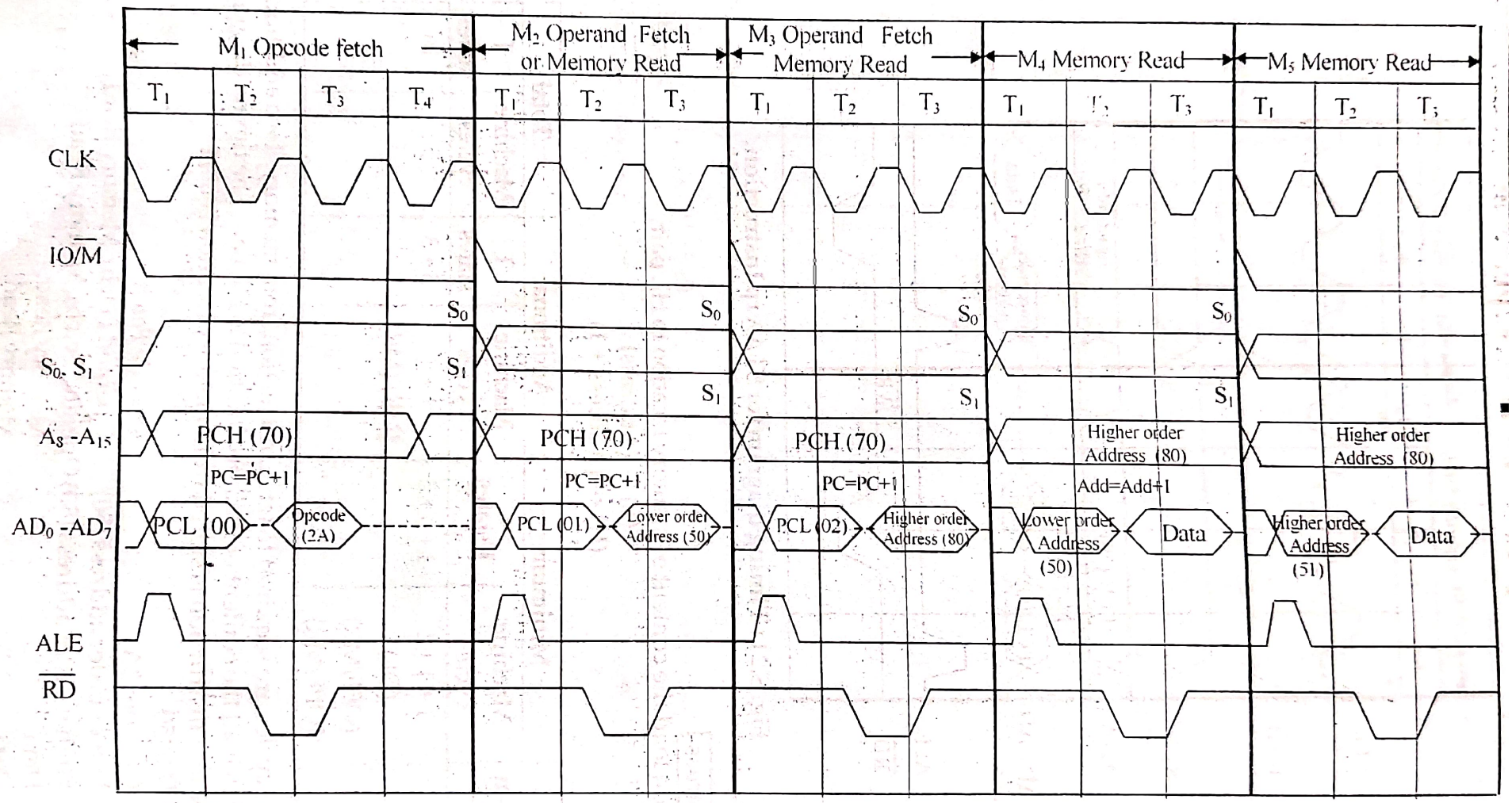


Fig. 8.22 : Timing diagram for LHLD address instruction.